

CCAF Practice Exam - Combined Questions

Total Questions: 234

Q1. Multi-Agent Research - Coordinator Next Step After Agents Complete You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. The web search and document analysis agents have both completed their tasks and returned findings to the coordinator. What is the appropriate next step for producing an integrated research output?

- A) Each agent directly sends its findings to the report generation agent, bypassing the coordinator
- B) The coordinator passes both sets of findings to the synthesis agent for unified integration
- C) The document analysis agent requests the web search results and merges them internally
- D) The coordinator concatenates the raw outputs from both agents and returns them as the final result

Answer: B

Q2. Multi-Agent Research - Subagent Error Handling for PDF Failures You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports. The document analysis subagent frequently encounters failures when processing PDF files ? some have corrupted sections causing parsing exceptions, others are password-protected, and occasionally the parsing library times out on large files. Currently, any exception immediately terminates the subagent and returns an error to the coordinator, which must decide whether to retry, skip the document, or fail the entire research task. This is causing excessive coordinator involvement in routine error handling. What's the most effective architectural improvement?

- A) Have the coordinator validate all documents before dispatching to the subagent, rejecting documents likely to cause failures to ensure the subagent only receives processable files.
- B) Create a dedicated error-handling agent that monitors all subagent failures via a shared queue and makes recovery decisions independently, dispatching retry commands directly to subagents.
- C) Have the subagent implement local recovery for transient failures and only propagate errors it cannot resolve to the coordinator, including what was attempted and any partial results obtained.
- D) Configure the subagent to always return partial results with success status, embedding error details in metadata. The coordinator treats all responses as successful and filters problematic items during synthesis.

Answer: C

Q3. Structured Data Extraction - Reliable JSON Schema Compliance You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high edge cases gracefully and integrate with downstream systems. Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

- A) Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.
- B) Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.
- C) Append instructions like "Output only valid JSON matching the schema exactly?" and implement retry logic to re-prompt when JSON parsing fails.
- D) Define a tool with your target schema as input parameters and have Claude call it with the extracted data.

Answer: D

Q4. Structured Data Extraction - Skills Schema Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like `?Python and SQL?` are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says `?Extract all skills mentioned.?` What's the most effective improvement?

- A) Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- B) Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- C) Add constraints: `?Extract 10-20 skills maximum, one skill per entry, only explicitly named skills.?`
- D) Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

Answer: A

Q5. Structured Data Extraction - Restaurant Menus Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting ? prices as `?$12?` vs `?$12.00?`, dietary info as icons vs text. What's the most reliable approach?

- A) Request multiple extraction attempts per document and select the most common format.
- B) Define a strict output schema and include format normalization rules in your prompt.
- C) Extract data as-is and normalize formats in post-processing code after Claude returns.
- D) Use separate extraction calls for each field to ensure consistent handling of each type.

Answer: B

Q6. Structured Data Extraction - Document Type Classification The extraction pipeline receives documents of varying types ? some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own JSON schema tailored to the document type. During testing, you observe that with `tool_choice: "auto"`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- A) Consolidate all document types into a single unified-schema extraction tool and force that tool.
- B) Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.
- C) Keep `tool_choice: "auto"` with system prompt instructions requiring tool use.
- D) Set `tool_choice: "any"` with all extraction tools defined.

Answer: B

Q7. Structured Data Extraction - Resume Extraction The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude's output consistently matches the schema?

- A) Define a tool with an input schema matching your required JSON structure and extract the data from Claude's `tool_use` response.
- B) Make two separate API calls ? first extracting information as text, then asking Claude to format that text as JSON.
- C) Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.
- D) Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.

Answer: A

Q8. Structured Data Extraction - Batch Processing Strategy Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema validation on the first attempt, while the remaining 18% fail due to diverse issues ? missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need 2-3 prompt refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

A) Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.

B) Split documents into 10 sequential batches of 5,000 each, analyzing results and refining prompts between batches to improve extraction quality progressively.

C) Submit all 50,000 documents via batch API, then submit failed extractions in successive batches ? refining prompts between each batch ? until all documents pass validation.

D) Process 2,000 sample documents via real-time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.

Answer: D

Q9. Structured Data Extraction - High-Confidence Error Detection The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions (>85%) also contain errors ? cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

A) Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.

B) Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.

C) Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.

D) Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.

Answer: A

Q10. Structured Data Extraction - Inconsistent Source Values Monitoring reveals that specifications sometimes appear inconsistently within source documents. For example, a summary section might state ?Battery: 4000mAh? while the detailed specs table shows ?Battery: 4200mAh.? Your current schema uses a single `battery_capacity_mah` field. This inconsistency occurs in approximately 15% of documents, and historical analysis confirms the detailed specs table is more accurate 90% of the time. What's the most effective approach?

A) Include extraction instructions specifying to prefer values from the detailed specs table when multiple values exist, keeping the single-value schema.

B) Implement schema validation that rejects results containing conflicting values, requiring source document correction before processing.

C) Add a `conflict_detected` boolean field that flags inconsistencies, triggering manual review for affected documents.

D) Change to an array field that captures all values found with their source locations, letting downstream systems apply precedence rules.

Answer: A

Q11. Structured Data Extraction - Informal Measurements Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like ?a handful? or ?a splash,? the model either invents specific amounts or leaves fields empty ? accounting for 23% of all corrections. How should you use this feedback to improve extraction accuracy?

- A) Update your JSON schema to add a ?measurement_type? enum field (precise/informal).
- B) Add few-shot examples to your prompt demonstrating correct handling of informal measurements ? extracting them verbatim rather than converting or omitting them.
- C) Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty.
- D) Fine-tune the model on the 847 corrected extractions.

Answer: B

Q12. Structured Data Extraction - Processing Pipeline Architecture Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- A) Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- B) Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.
- C) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.
- D) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

Answer: C

Q13. Structured Data Extraction - Hallucination Prevention with Optional Fields Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema?s required fields. For example, a document mentioning only dimensions receives a fabricated ?weight: 2.3 kg? in the extraction output. What schema design change most effectively addresses this hallucination behavior?

- A) Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.
- B) Change fields that may not exist in source documents from required to optional, allowing the model to omit them.
- C) Add explicit instructions to the prompt stating ?only extract information explicitly stated in the document; use placeholder text for missing values?
- D) Add a ?confidence? field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.

Answer: B

Q14. Structured Data Extraction - Inconsistent Material Extraction Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the ?materials? field ? sometimes returning ?cotton blend?, other times ?Cotton/Polyester mix?, and occasionally omitting the field when material information is clearly present in the source. What?s the most effective way to improve extraction consistency?

- A) Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability.
- B) Make the ?materials? field required instead of optional in the schema to force the model to always extract a value.
- C) Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats.
- D) Set temperature to 0 to eliminate randomness and ensure deterministic outputs.

Answer: C

Q15. Structured Data Extraction - Tool Ordering Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like `?extract the metadata and tell me how cited it is,` Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- A) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- B) Set `tool_choice` to `?any?` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- C) Set `tool_choice` to `?auto?` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- D) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

Answer: A

Q16. Structured Data Extraction - Semantic Validation Failures Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation ? for example, line item amounts don't sum to the extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review. What's the most effective approach to reduce manual review volume while maintaining accuracy?

- A) Retry the extraction up to 3 times when validation fails, accepting the first result that passes validation.
- B) When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction.
- C) Implement post-processing logic that automatically corrects common errors, such as recalculating totals from line items when sums don't match.
- D) Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially.

Answer: B

Q17. Structured Data Extraction - Context Length Exceeded in Batch After your daily batch of 10,000 documents completes, 300 documents (3%) failed with `?context_length_exceeded?` errors. The results file identifies each failure by `custom_id`. What's the most cost-effective approach to process these failures?

- A) Resubmit the entire 10,000 document batch using a model tier with a larger context window.
- B) Increase the `max_tokens` parameter for the 300 failed documents and resubmit them in a new batch.
- C) Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions.
- D) Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts.

Answer: C

Q18. Agent - Context Degradation in Game Engine Exploration Your agent has spent 25 minutes exploring a game engine's rendering subsystem ? reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference `?typical rendering patterns?` rather than the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier. What's the most effective approach?

- A) Continue in the current context with more targeted prompts referencing the specific classes by name.
- B) Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.
- C) Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- D) Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's `CLAUDE.md`.

Answer: B

Q19. Agent - Finding Error Source in Codebase An engineer sees an unfamiliar error message `?SYNC_CONFLICT: entity version mismatch detected?` in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- A) Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- B) Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- C) Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.
- D) Use Grep to search for distinctive text from the error message (like `?SYNC_CONFLICT?` or `?entity version mismatch?`), then Read the matching files to understand context.

Answer: D

Q20. Agent - Exploring Two Refactoring Approaches An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth ? having the agent propose specific code changes for each ? before deciding which to implement. What's the most effective way to structure this exploration?

- A) Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- B) Use `/fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- C) Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.
- D) Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Answer: B

Q21. Agent - Comparing Testing Strategies Your agent has analyzed a complex service module ? reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- A) Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.
- B) Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- C) Export the analysis session's key findings to a file, then create two new sessions that reference this file.
- D) Start two fresh sessions, having each re-read the relevant source files before beginning.

Answer: A

Q22. Agent - Security Scanning Tool Choice You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like `eval()` across a large codebase, which tool should your agent use for content search?

- A) Use Grep to search for the pattern `?eval(? across all files in the codebase.`
- B) Use Bash to run `ls -R | grep eval` to recursively list files containing `eval`.
- C) Use Glob with a pattern like `**/eval*` to find files, then Read each matching file.
- D) Read the project's main entry file and follow import statements to trace where `eval` might be used.

Answer: A

Q23. Agent - Resuming a Named Session An engineer used Claude Code yesterday to investigate authentication flows in a legacy monolith, building up significant context over a 2-hour session. Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was named `auth-deep-dive`. How should she resume?

- A) Use `--session-id` with the UUID from yesterday's session transcript file.
- B) Use `--continue` to pick up where the most recent conversation left off.
- C) Start fresh and re-read the same files.
- D) Use `--resume auth-deep-dive` to load that specific session by name.

Answer: D

Q24. Agent - Prioritizing Test Coverage An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize. How should the agent decompose this open-ended task?

- A) Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning.
- B) Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.
- C) Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.
- D) Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or business importance.

Answer: B

Q25. Agent - Multi-Phase Workflow Benefit An engineer submits two requests: - Request A: `Rename the getUserData function to fetchUserProfile everywhere it's used.` - Request B: `Improve error handling throughout the data processing module - add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data.` For which request does specifying an explicit multi-phase workflow (such as `analyze -> propose -> implement with review`) most improve outcome quality?

- A) Request A, the function rename task
- B) Request B, the error handling task
- C) Neither request benefits significantly
- D) Both requests benefit equally

Answer: B

Q26. Agent - MCP Tool Adoption After adding an MCP server with specialized code refactoring tools (`extract_function`, `rename_variable`, `inline_function`), you notice the agent still uses basic text manipulation via `Write` and `Bash` `sed` commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like `extract_function: Extracts a function from code.` What's the most effective way to improve adoption of the MCP refactoring tools?

- A) Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- B) Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.
- C) Accept this as expected behavior since simpler tools like `sed` are more predictable than specialized refactoring tools.
- D) Remove the `Write` tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.

Answer: A

Q27. Agent - Resuming Session with Changed Files Your code exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

- A) Resume the session without informing the agent about the changed files.
- B) Start a fresh session to ensure the agent works with current codebase state without stale assumptions.
- C) Resume the session and inform the agent which specific files changed for targeted re-analysis.
- D) Resume the session and immediately have the agent re-read all 12 previously analyzed files.

Answer: C

Q28. Agent - Context Degradation in Long Sessions During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

- A) Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.
- B) Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.
- C) Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.
- D) Create summaries of all source files before exploration begins, loading only these compressed representations into context.

Answer: B

Q29. Agent - MCP Server Tool Access You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to 'create a branch for JIRA-123 and add documentation links to the ticket,' how does the agent access tools across these servers?

- A) The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- B) Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- C) You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- D) The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.

Answer: B

Q30. Agent - Context Pressure in Large Codebase Investigation An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate - it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows. What's the most effective approach to complete this investigation?

- A) Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.
- B) Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.
- C) Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.
- D) Spawn subagents to investigate specific questions (e.g., 'find all test files for payment processing,' 'trace refund flow dependencies') while the main agent coordinates findings and preserves high-level understanding.

Answer: D

Q31. Agent - Investigating 500 Errors Across Layers A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers. What task decomposition approach would be most effective?

A) Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates.

B) Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges.

C) Define a fixed sequence of investigation steps upfront ? grep for error patterns, then read error handlers, then check database queries, then examine middleware ? executing each step regardless of intermediate findings.

D) Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading.

Answer: B

Q32. Agent - Caching Layer Investigation with Context Constraints An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

A) Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.

B) Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.

C) Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

D) Use Grep to search for ?invalidate? and ?expire? patterns across all files, then Read only those specific line ranges with minimal surrounding context.

Answer: C

Q33. Multi-Agent Research - Citation Attribution In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

A) Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

B) Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

C) Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

D) Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

Answer: A

Q34. Multi-Agent Research - Slow Summarization Requests Production monitoring shows that follow-up queries like ?summarize what we learned about market trends? consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

A) Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.

B) Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

C) Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis.

D) Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.

Answer: C

Q35. Multi-Agent Research - Incomplete Coverage The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis agents with targeted queries before invoking synthesis again.

B) Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

C) Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

D) Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

Answer: A

Q36. Multi-Agent Research - Synthesis Format Handling A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

A) Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation

B) Update the synthesis agent to render each content type appropriately—financial data as tables, news as prose

C) Standardize all subagent outputs to prose summaries with inline citations

D) Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence

Answer: B

Q37. Multi-Agent Research - Document Analysis Gaps in Strict Pipeline Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps—for instance, noting “the retrieved sources discuss API authentication but lack details on token refresh patterns”—but under the current strict pipeline, this insight isn't actionable since search has already completed. What's the most effective architectural change?

A) Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

B) Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

C) Add a research planning agent before the search phase that decomposes topics into specific sub-questions.

D) Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

Answer: B

Q38. Multi-Agent Research - Synthesis Agent Cannot Complete Task After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

A) The subagents need to share a single API connection to enable automatic context sharing between invocations.

B) The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

- C) The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.
- D) The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Answer: D

Q39. Multi-Agent Research - Source Attribution Lost During Summarization The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations?the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

- A) Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.
- B) Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.
- C) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).
- D) Have the report generator query the web search agent to re-locate sources for claims in the final report.

Answer: C

Q40. Multi-Agent Research - State Management After Pipeline Crash Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

- A) Have each agent maintain its own persistent state file and reload it independently at the start of each session.
- B) Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.
- C) Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.
- D) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

Answer: D

Q41. Multi-Agent Research - Inconsistent Uncertainty Handling Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate \$50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates \$35B (±\$7B, 95% CI)", the coordinator either picks one arbitrarily or produces vague statements like "the market may be \$35B-\$50B depending on factors." What systematic approach best addresses this?

- A) Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.
- B) Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context.
- C) Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.
- D) Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence.

Answer: B

Q42. Multi-Agent Research - Providing Context to Synthesis Subagent After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

- A) Include the complete findings from both subagents directly in the synthesis subagent's prompt
- B) Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator
- C) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results
- D) Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

Answer: A

Q43. Multi-Agent Research - Optimizing for Query Complexity In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- A) Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.
- B) Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.
- C) Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- D) Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

Answer: A

Q44. Multi-Agent Research - Context Passing Strategy After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

- A) Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.
- B) Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.
- C) Pass the full accumulated context from all prior agents.
- D) Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.

Answer: D

Q45. Multi-Agent Research - Temporal Data Interpretation When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2021: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

- A) Configure the web search agent to only return results from the past 6 months.
- B) Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.
- C) Require subagents to include publication or data collection dates in their structured outputs.
- D) Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

Answer: C

Q46. Multi-Agent - Coordinator Delegation Not Executing The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate?it generates messages like ?I'll ask the web search agent to find sources on this topic??but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

- A) The coordinator?s allowedTools configuration doesn?t include ?Task?, so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.
- B) The AgentDefinitions are configured correctly, but the coordinator?s system prompt doesn?t explicitly list the available subagent types, preventing the model from knowing they can be invoked.
- C) Subagent context isolation means task descriptions from the coordinator don?t automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.
- D) The coordinator?s max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

Answer: A

Q47. Multi-Agent - Running Subagents Concurrently Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent?neither requires the other?s output. How should you modify the system to run these subagents concurrently?

- A) Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator-subagent pair, then aggregates results.
- B) Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.
- C) Add detailed instructions to the coordinator?s system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time.
- D) Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time.

Answer: B

Q48. Automated Code Review - Missing Genuine Bugs Your automated code review is missing genuine bugs in pull requests. Investigation reveals that your review prompt includes the instruction: ?Only flag critical issues that would definitely cause production failures. Ignore minor concerns and anything you're uncertain about.? Developers confirm that some missed bugs are real logic errors the model investigated but chose not to report. The team requires that review output remain structured (each finding tagged with metadata) and actionable. Which prompt change both removes the cause of the suppressed findings and preserves structured, tagged output for downstream filtering?

- A) Enable extended thinking and instruct the model to reason step-by-step about each code change before producing its review.
- B) Remove all severity-related instructions from the prompt and let the model use its default judgment about what to report.
- C) Instruct the model to report all findings with a confidence level and severity tag, deferring filtering to a downstream step.
- D) Add a second review pass that re-reads the diff using the same prompt, looking for anything the first pass may have missed.

Answer: C

Q49. Automated Code Review - Reducing False Positives After deploying automated code review, developers report that approximately 35% of flagged findings are false positives falling into consistent patterns: style suggestions contradicting team conventions, security warnings for patterns safe in your deployment context, and performance suggestions that would degrade your specific use case. You want to reduce false positives while maintaining the ability to catch genuine issues. Which approach best enables the model to generalize its judgment to novel code patterns it hasn't seen before?

- A) Add instructions to your system prompt to ?be conservative,? ?only flag definite issues,? and ?consider that some patterns may be intentional.?
- B) Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine

issues in each category.

C) Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off."

D) Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt.

Answer: B

Q50. Automated Code Review - Reducing Dismissals Your automated review generates many findings per PR, but developer feedback shows roughly half are dismissed as "not worth addressing." Analysis reveals dismissed findings are often technically accurate but involve minor style preferences or patterns acceptable in your codebase. Before adding infrastructure complexity, what prompt design change would most effectively reduce dismissals while maintaining detection of genuine issues?

A) Implement a secondary classification model that filters Claude's findings based on predicted developer acceptance

B) Add instructions asking Claude to rate each finding's confidence (1-10) and only include findings rated 8 or higher

C) Append instructions to "only report findings you are highly confident are genuine problems"

D) Add explicit criteria defining which issues to report (bugs, security) versus skip (minor style, local patterns)

Answer: D

Q51. Claude Code Review - Reading Surrounding Files Your pipeline runs: `claude -p \ --system-prompt "$PROMPT" \ < diff.txt` The reviews complete and return feedback, but Claude only comments on the piped diff text?it never reads surrounding files in the checked-out repository to understand broader context, even when the diff modifies a function called by many other modules. Which change to the invocation will cause Claude to read related files in the repository while still applying your custom review instructions?

A) Stop piping the diff via stdin and instead embed the diff contents inside the prompt string, so Claude Code treats the invocation as an agentic session rather than a stream-processing one.

B) Keep `--system-prompt` and add `--allowedTools "Read,Glob,Grep"` so that the non-interactive `-p` mode permits file system tools that it otherwise disables.

C) Replace `--system-prompt` with `--append-system-prompt` so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools.

D) Remove `--system-prompt` entirely and place the review instructions in a `CLAUDE.md` file at the repo root, since `--system-prompt` is incompatible with tool use under `-p`.

Answer: C

Q52. API Cost Optimization - Message Batches API Your pipeline includes a release notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes aren't needed until the following morning (results have ~12 hours of acceptable latency). Your team needs to reduce per-token API cost for this step while keeping the same model and prompts (no change to model tier or output quality). Which approach satisfies all of these constraints?

A) Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, since fewer requests always reduces total token cost.

B) Submit the 200 requests to the Message Batches API with unique `custom_ids` and retrieve results once the batch ends, which applies a 50% discount to all input and output tokens.

C) Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.

D) Issue the 200 Messages API requests in parallel using concurrent connections, since concurrency lowers the per-token price charged by the API.

Answer: B

Q53. Code Review Tool - Analyzing Dismissals You built an LLM-powered code review tool that analyzes pull requests and outputs structured findings. Each finding is a JSON object with `file_path`, `line_number`, `issue_category` (e.g., `?security?`, `?style?`), and `description`. Developers can click `?dismiss?` on any finding they consider unhelpful. Currently 35% of findings get dismissed. You want to analyze these dismissals to understand what your system is getting wrong and improve your prompts accordingly. What change to your output structure would best support this analysis?

- A) Expand the `description` field to include more detailed explanations of why each issue matters and how to fix it.
- B) Add a `detected_pattern` field recording the code construct (e.g., `?single-letter loop variable?`) that triggered each finding.
- C) Remove the `issue_category` field and track dismiss rates at the individual finding level only.
- D) Add a `model_confidence` field (0.0-1.0) and filter out findings below a threshold calibrated against historical dismiss rates.

Answer: B

Q54. Code Review Pipeline - Reducing Noise from Generated Code The automated review pipeline generates an average of 16 findings per PR. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your linter in CI, (2) findings on auto-generated template code under `src/gen/`, and (3) rendering helper patterns that are intentional project conventions but get flagged because they resemble common anti-patterns. Only about 4 findings per PR are genuine logic bugs. What is the most effective way to reduce this noise while preserving detection of real issues?

- A) Create a `REVIEW.md` at the repository root with skip rules for CI-enforced checks and generated files, and a verification requirement that rendering-related findings must cite a specific line demonstrating incorrect behavior.
- B) Configure separate GitHub Action workflow files for each code area—one for generated code with findings suppressed, one for rendering code with custom review instructions, and a general workflow for everything else.
- C) Add the `prompt` parameter to the GitHub Action configuration with instructions to suppress lint-duplicate findings, ignore generated template code, and apply stricter evidence requirements for rendering-related issues.
- D) Add detailed explanations to the project's `CLAUDE.md` describing which patterns are intentional, that linting is handled separately by CI, and that the `src/gen/` directory contains auto-generated template code.

Answer: D

Q55. Code Review - Branch-Level Coverage Gaps Your code review prompts include both implementation changes and the corresponding test file, but the LLM's review comments fail to point out untested code paths. Analysis reveals the model correctly flags functions that have no tests at all, but fails to identify when conditional branches or error-handling paths within tested functions lack coverage. What's the most effective way to improve detection of branch-level coverage gaps without overcomplicating the pipeline?

- A) Include few-shot examples showing code with an uncovered branch paired with the review comment identifying the specific missing test case
- B) Restructure the prompt to interleave implementation and tests, presenting each function followed immediately by its test cases
- C) Implement a multi-pass pipeline where separate LLM calls first extract all conditional branches, then cross-reference each against test assertions in a second pass
- D) Add explicit instructions directing the model to enumerate each conditional branch and exception path, then verify each has a corresponding test assertion

Answer: D

Q56. Claude Code Review - Capping Iterations and Cost During initial testing of the automated review pipeline, you notice that reviews on large PRs (50+ changed files) sometimes take over 20 minutes and cost \$8-12 per run due to extensive agentic loops. Claude reads files, runs analysis tools, and iterates many times. Your team needs each invocation to abort once it reaches a fixed iteration count and a fixed dollar amount, enforced by Claude Code itself rather than the surrounding job runner. Which configuration change directly enforces both of those per-invocation caps?

- A) Set `--permission-mode compact` to auto-deny any tool permission requests not in the explicitly allowed set.
- B) Set `timeout-minutes: 5` on the GitHub Actions job step and monitor per-run costs via the Anthropic Console usage dashboard.
- C) Add `--max-turns 18 --max-budget-usd 2.00` to the `claude -p` invocation to cap iterations and spend.

D) Switch the --model flag to a smaller, cheaper model so each iteration uses fewer tokens and lower per-call cost.

Answer: C

Q57. Code Review - Cross-File Bug Detection Your pipeline reviews every PR using a single API call with a static prompt containing the diff and full text of each changed file?unchanged files are not included. Reviews are posted asynchronously and don't block PR creation. Developers report that reviews consistently miss bugs involving cross-file interactions?for example, a PR renames a function?s parameters but the review doesn?t flag callers in unchanged files that still use the old argument order. Evaluation shows cross-file bugs account for 35% of production incidents from reviewed PRs. What is the most effective change to your review design?

A) Use static analysis to build a dependency graph of changed code, then expand the prompt to include all files within two dependency hops of any changed file.

B) Run parallel review passes per changed file with direct dependents included in each pass, then aggregate and deduplicate findings using a final summarization call.

C) Redesign the review as a turn-limited agentic task where the model can read files and search the codebase via tools, following references to verify cross-file findings.

D) Add chain-of-thought instructions asking the model to list all external references in the diff, then reason step-by-step about how each change might affect callers in other files.

Answer: C

Q58. Code Review - Truncated JSON Output Your automated review calls the Claude API for each PR, using tool_use with a report_findings tool that returns a JSON array of finding objects (each with file_path, line_number, severity, category, and description). During testing on a large PR touching 30+ files, the response hits the max_tokens limit and the output is truncated mid-JSON, causing your pipeline?s parser to fail. What is the most effective way to handle this?

A) Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings.

B) Switch from tool_use to prompting Claude to return findings as a markdown list.

C) Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.

D) Increase max_tokens to the model?s maximum and instruct Claude to keep finding descriptions under 50 words each.

Answer: C

Q59. Test Generation - Reducing Low-Value Tests Your test generation produces unit tests for new code, but reviews show 55% are low-value: trivial assertions that only verify functions don?t throw exceptions, tests duplicating existing coverage, or tests ignoring your team?s fixture conventions. How do you reduce the rate of low-value tests being generated in the first place?

A) Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers.

B) Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions.

C) Add post-generation coverage analysis that automatically filters out any generated test that doesn?t increase line coverage beyond what existing tests provide.

D) Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing.

Answer: B

Q60. Code Review - Improving Recall Without Increasing False Positives After deploying the automated review, you notice high precision but low recall?real bugs are slipping through undetected. Investigation reveals your review prompt instructs Claude to ?only report high-confidence issues you are certain about? and ?err on the side of not commenting.? Developers appreciate the low noise, but a race condition that caused a production outage was visible in a reviewed PR and went unreported. You need to substantially improve bug detection while keeping false positive rates manageable for your team. What is the most effective approach?

- A) Add detailed few-shot examples demonstrating bug categories Claude should flag?race conditions, null dereferences, error handling gaps?while keeping the high-confidence filtering instruction to maintain current precision levels.
- B) Split the review into a finding stage where Claude?s goal is coverage?flagging every potential issue with confidence and severity metadata?and a separate stage that thresholds those findings.
- C) Expand the context window by including related test files, recent git history, and the module?s dependency graph alongside the diff, giving Claude richer signals to assess issue severity.
- D) Remove the conservative filtering instructions and prompt Claude to report all potential issues, then apply a programmatic filter to deduplicate and suppress categories that historically generate false positives.

Answer: B

Q61. Code Review - Recall Trade-off Between Categories Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall for API design findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41% but API design recall drops to 68%. How should you address this trade-off to improve detection across both categories?

- A) Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculations and boundary conditions in grading thresholds.
- B) Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic patterns.
- C) Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.
- D) Split the review into separate focused prompts?one for security and API design, another for business logic?each with dedicated examples, then consolidate findings before posting.

Answer: D

Q62. Structured Data Extraction - Context Window and Accuracy Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model?s context window is 200K tokens. What is the most likely cause?

- A) Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.
- B) The model distributes attention proportionally across input length, causing fields mentioned only once near the document?s end to receive insufficient processing focus.
- C) Very long documents exceed the model?s effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.
- D) Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.

Answer: A

Q63. Structured Data Extraction - High-Confidence Errors The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high confidence extractions (>85%) also contain errors?cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time. What approach is most effective?

- A) Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.
- B) Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.
- C) Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.
- D) Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.

Answer: C

Q64. Structured Data Extraction - Long Meeting Transcripts Evaluation shows 94% extraction accuracy on short meeting transcripts (<30 minutes) but only 68% on longer transcripts (>60 minutes) where discussions meander and key information is scattered throughout. Transcripts of both lengths fit within the model?s context window. What pattern most effectively improves accuracy on complex, lengthy documents?

- A) Split lengthy transcripts into chunks, extract from each chunk separately, then merge and deduplicate the results.
- B) Add a pre-extraction step where the model summarizes key discussions and conclusions before performing structured extraction.
- C) Add few-shot examples demonstrating correct extraction from lengthy meetings with scattered information.
- D) Upgrade to a more capable model tier for the extraction task.

Answer: B

Q65. Structured Data Extraction - Reducing False Extractions Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article?for example, outputting ?500? for attendee_count when the source contains no attendance information. What?s the most effective way to reduce these false extractions?

- A) Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.
- B) Add prompt instructions to return null for any field where information is not directly stated in the source.
- C) Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- D) Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

Answer: B

Q66. Structured Data Extraction - Schema Conformance from Resumes The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude?s output consistently matches the schema?

- A) Define a tool with an input schema matching your required JSON structure and extract the data from Claude?s tool_use response.
- B) Make two separate API calls?first extracting information as text, then asking Claude to format that text as JSON.
- C) Parse Claude?s text response with regex patterns to extract JSON objects, using retry logic for malformed responses.
- D) Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.

Answer: A

Q67. Structured Data Extraction - Hallucinated Values for Required Fields Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated weight: 2.3 kg in the extraction output. What schema design change most effectively addresses this hallucination behavior?

- A) Add a confidence field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.
- B) Change fields that may not exist in source documents from required to optional, allowing the model to omit them.
- C) Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values."
- D) Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.

Answer: B

Q68. Structured Data Extraction - Validation Before Automation Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence ≥90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- A) Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.
- B) Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- C) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- D) Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

Answer: C

Q69. Structured Data Extraction - Document Type Routing with tool_choice The extraction pipeline receives documents of varying types—some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own JSON schema tailored to the document type. During testing, you observe that with tool_choice: "auto", Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- A) Keep tool_choice: "auto" with system prompt instructions requiring tool use.
- B) Set tool_choice: "any" with all extraction tools defined.
- C) Consolidate all document types into a single unified-schema extraction tool and force that tool.
- D) Add a preliminary classification call, then make a second call with tool_choice forced to the identified extraction tool.

Answer: B

Q70. Structured Data Extraction - Invoice Line Items vs Grand Total Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 16% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What's the most effective approach to improve extraction reliability?

- A) Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.
- B) Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.
- C) Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.
- D) Add a calculated_total field where the model sums extracted line items alongside a stated_total field. Flag records for human review when values differ.

Answer: D

Q71. Structured Data Extraction - Tool Ordering and Enrichment Pipeline Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- A) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- B) Set `tool_choice` to "any" so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- C) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- D) Set `tool_choice` to "auto" and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.

Answer: C

Q72. Structured Data Extraction - Batch vs Real-Time Processing Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- A) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- B) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.
- C) Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- D) Submit all documents to the Batch API with `custom_ids` for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

Answer: B

Q73. Structured Data Extraction - Empty Required Fields After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats: inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective way to address these failures?

- A) Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- B) Add few-shot examples demonstrating extractions from documents with varied structures, showing how to identify citations in different formats and locate methodology details across section types.
- C) Implement retry logic that re-sends requests when validation detects empty required fields.
- D) Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.

Answer: B

Q74. Structured Data Extraction - Semantic Errors and Reviewer Allocation After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions. Which approach most effectively allocates reviewer attention?

- A) Prioritize review of all extractions where required fields are empty or explicitly marked as not found.
- B) Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.
- C) Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.
- D) Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.

Answer: C

Q75. Structured Data Extraction - Pydantic Validation Failures Monitoring shows 12% of extractions fail Pydantic validation with specific errors like "expected float for quantity, got 2 to 3". Retrying these requests without modification produces identical failures. What's the most effective approach to recover from these validation failures?

- A) Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation.
- B) Set temperature to 0 to eliminate output variability and ensure consistent formatting.
- C) Pre-process source documents to standardize problematic formats before sending them for extraction.
- D) Send a follow-up request including the validation error, asking the model to correct its output.

Answer: D

Q76. Structured Data Extraction - Inconsistent Specifications in Source Documents Monitoring reveals that specifications sometimes appear inconsistently within source documents. For example, a summary section might state "Battery: 4000mAh" while the detailed specs table shows "Battery: 4200mAh." Your current schema uses a single battery_capacity_mah field. This inconsistency occurs in approximately 15% of documents, and historical analysis confirms the detailed specs table is more accurate 90% of the time. What's the most effective approach?

- A) Include extraction instructions specifying to prefer values from the detailed specs table when multiple values exist, keeping the single-value schema.
- B) Change to an array field that captures all values found with their source locations, letting downstream systems apply precedence rules.
- C) Implement schema validation that rejects results containing conflicting values, requiring source document correction before processing.
- D) Add a conflict_detected boolean field that flags inconsistencies, triggering manual review for affected documents.

Answer: A

Q77. Claude Code - Iterating on Data Migration Script You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields. What's the most effective way to iterate toward a working solution?

- A) Provide a test case with example input containing null values and the expected output, then ask Claude to fix it.
- B) Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.
- C) Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.
- D) Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.

Answer: A

Q78. Claude Code - Critical Bug Fix Approach A critical bug is affecting production users. Error logs show exceptions in the OrderProcessing module with a clear stack trace pointing to a specific function. You haven't worked with this module before. What's the most effective approach?

- A) Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause.
- B) Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes.
- C) Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically.
- D) Enter plan mode to explore the module's architecture and dependencies before attempting any fixes.

Answer: A

Q79. Claude Code - Inconsistent Error Handling Conventions You've documented API error handling conventions in a CLAUDE.md file at your project root, specifying that endpoint handlers should use a custom ApiError class. After several sessions, you notice Claude Code sometimes follows these conventions and sometimes uses generic try/catch blocks with string messages. The inconsistency appears random across different coding sessions. What's the most efficient first diagnostic step?

- A) Add more detailed code examples to your CLAUDE.md showing the exact ApiError usage pattern for different endpoint types.
- B) Run /memory to check which memory files are loaded and verify your CLAUDE.md is included.
- C) Search for conflicting instructions in ~/.claude/CLAUDE.md or ~/.claude/rules/ that might override your project conventions.
- D) Create path-specific rules in .claude/rules/handlers.md with YAML frontmatter scoping the error handling instructions to your API handler files.

Answer: C

Q80. Claude Code - CLAUDE.md Organization for Multi-File-Type Repository Your infrastructure-as-code repository includes Terraform modules (/terraform/), Kubernetes manifests (/kubernetes/), and CI/CD pipeline scripts (/pipelines/). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens. What is the best approach to reorganize to only relevant guidance loads when editing specific file types?

- A) Split content into subdirectory CLAUDE.md files (/terraform/CLAUDE.md, /kubernetes/CLAUDE.md, etc.) so Claude loads directory-specific guidance.
- B) Keep the root CLAUDE.md and use @imports/import syntax to modularly include tool-specific guidance files from separate documents.
- C) Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., ### Terraform Conventions?), improving organization and readability.
- D) Create files in .claude/rules/ with YAML frontmatter path-scoping (e.g., path: /terraform/**), loading rules only when editing matching files.

Answer: D

Q81. Claude Code - MCP Server Prompts Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as deploy_checklist and incident_response) in addition to tools. How do these MCP prompts become accessible within Claude Code?

- A) They appear as slash commands (e.g., /mcp_servername_deploy_checklist) that you can invoke, with arguments passed after the command name.
- B) They are surfaced as @-mentionable resources alongside files, fetched and attached to your message when referenced.
- C) They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.
- D) They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

Answer: A

Q82. Claude Code - Configuration Mechanisms for Team Requirements Your team has three requirements for Claude Code's behavior in your project: 1. Claude must never modify files in the db/migrations/ directory 2. Claude should prefer your custom logging module over console.log 3. All TypeScript files must be auto-formatted with Prettier after every edit All three are currently written as instructions in your project's CLAUDE.md. During a complex refactoring session, a developer discovers that Claude edited a migration file, violating requirement #1. How should you restructure these requirements across Claude Code's configuration mechanisms?

- A) Rewrite all three requirements in CLAUDE.md using stronger directive language and add few-shot examples that demonstrate Claude refusing to edit migration files and running Prettier after edits.
- B) Configure hooks for all three: a PreToolUse hook script that blocks Edit calls targeting db/migrations/, a PreToolUse hook script that adds logging convention context before edits, and a PostToolUse hook that runs Prettier after TypeScript edits.

C) Add `Edit:!/db/migrations/**` to `permissions.deny` in the project settings, keep the logging preference in `CLAUDE.md`, and add a `PostToolUse` hook on the `Edit` tool that runs Prettier on changed TypeScript files.

D) Move all three requirements into `.claude/rules/` as path-scoped rules: one targeting `db/migrations/**` that forbids editing those files, and others targeting `**/*.ts` for the logging convention and formatting instruction.

Answer: C

Q83. Claude Code - Shared Skill File Placement Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing `/migrate-component`. The workflow should stay in sync as the team iterates on it. Where should you place the skill file?

A) In `.claude/skills/migrate-component/SKILL.md` at the project root, committed to version control.

B) In the project's `.claude/settings.json` using a `skillOverrides` entry to register and define the workflow.

C) As a detailed instruction block in the project's root `CLAUDE.md` file.

D) In `~/.claude/skills/migrate-component/SKILL.md` on each developer's machine.

Answer: A

Q84. Claude Code - Following Existing Patterns for New Module You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging. You've identified three existing modules that exemplify these patterns: `db_utils.py`, `error_handlers.py`, and `audit_logger.py`. This is a one-off integration task; these patterns are well-documented in your team wiki and don't need additional project-level documentation. What's the most effective approach?

A) Add documentation of each pattern to your `CLAUDE.md` file, establishing them as project conventions that Claude will apply automatically.

B) Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module.

C) Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow.

D) Use `@` references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow.

Answer: D

Q85. Claude Code - Code Review Checklist Workflow Your team wants Claude to follow a detailed code review checklist (8 items covering API changes, test coverage, documentation, security, etc.) when reviewing pull requests. The team also uses Claude extensively for other tasks: writing new features, debugging production issues, and generating documentation. Currently, developers paste the checklist at the start of each review session. Which approach best addresses this workflow need?

A) Create a `/review` slash command containing the checklist, invoked when starting reviews.

B) Add the checklist to the project's `CLAUDE.md` file under a `?Code Review?` section.

C) Create a dedicated review subagent with the checklist embedded in its configuration.

D) Configure plan mode as the default for code review sessions.

Answer: A

Q86. Claude Code - Iterative Caching Implementation You're implementing a caching layer for API responses to speed up the `/products` endpoint. You have a rough idea: Redis with a 5-minute TTL, but you're new to production caching and aren't sure what other considerations a robust implementation requires. What's the most effective way to start your iterative workflow?

A) Write a specification with your known requirements and `?TBD?` markers for uncertain areas, having Claude propose solutions for each TBD as it implements.

B) Use plan mode to analyze the current `/products` endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.

C) Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes.

D) Start with a minimal request: ?Add Redis caching to /products with 5-minute TTL.? Add features and fix issues through follow-up prompts as problems surface during testing.

Answer: C

Q87. Claude Code - Iterating on PDF Report Formatting Issues You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact?changing column widths affects how dates render, and page breaks depend on content height. What's the most effective approach for iterating toward a working solution?

A) Start fresh with a detailed prompt specifying all formatting requirements upfront.

B) Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.

C) Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.

D) Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks?testing after each change.

Answer: D

Q88. Claude Code - Authentication Library Migration A security audit requires updating your authentication library from v2 to v3. The migration guide documents breaking changes: authenticate() now returns a Promise instead of accepting a callback, the user type has restructured fields, and three deprecated methods were removed. Grep shows the library is imported in 45 files across several modules. What's the most effective approach?

A) Update the dependency version, run the test suite, and use Claude Code to fix each failure as it appears.

B) Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration.

C) Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing.

D) Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files.

Answer: C

Q89. Claude Code - Enforcing Consistent Code Formatting Your team's CLAUDE.md includes a rule: ?Use 4-space indentation and always run Prettier formatting.? Despite this, code reviews reveal that roughly 30% of files Claude Code generates use inconsistent formatting?sometimes 2-space indentation, sometimes missing trailing commas. Adding emphasis (?IMPORTANT: You MUST use Prettier formatting?) reduces violations to about 15%, but doesn't eliminate them. What is the most effective way to ensure all generated code is consistently formatted?

A) Extract the formatting rules into a dedicated skill that Claude loads automatically when generating code, with more detailed examples of correct formatting.

B) Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations.

C) Configure a PostToolUse hook with an Edit/Write matcher that automatically runs Prettier on each file Claude modifies.

D) Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types.

Answer: C

Q90. Claude Code - Complex Graph Traversal Algorithm Implementation You're implementing a complex graph traversal algorithm with specific performance requirements and edge cases to handle (disconnected nodes, cycles, weighted edges). You want to structure your workflow for efficient iterative refinement with Claude. What approach will most effectively enable progressive improvement across multiple iterations?

- A) Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request.
- B) Provide Claude with a detailed natural language specification of the algorithm, including all requirements and edge cases. Review each output manually and provide descriptive feedback on what behavior needs to change.
- C) Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan.
- D) Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference.

Answer: A

Q91. Claude Code - Real-Time Updates Architecture Decision You're tasked with adding real-time updates to the application. This could be implemented using WebSockets, Server-Sent Events, or polling, each with different complexity, browser support, and infrastructure requirements. What's the most effective way to begin this task?

- A) Use direct execution to implement polling first, then evaluate whether to upgrade to WebSockets later.
- B) Enter plan mode to explore the architecture, evaluate trade-offs, and present options for team approval before implementing.
- C) Start direct execution with WebSockets, then refactor if infrastructure issues arise.
- D) Use direct execution with a prompt asking Claude to analyze all approaches and implement the one it determines is best.

Answer: B

Q92. Agentic Patterns - Agent Loop Turn Limit and Billing Disputes Production logs show that when the agent handles complex billing disputes requiring 6+ tool calls, it sometimes exhausts its max_turns limit after gathering data but before completing resolution or escalating. The team's goal is to guarantee that every customer interaction ends with either a completed resolution or a human handoff, regardless of how the agent loop terminates. Which approach achieves this guarantee?

- A) Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an automatic escalation once the agent reaches 80% of its max_turns limit.
- B) Add system prompt instructions telling the agent to call escalate_to_human with a summary of its findings whenever it determines it cannot complete resolution within its remaining actions.
- C) Split the workflow into two sequential agent invocations—a first agent gathers information via get_customer and lookup_order, then a second agent receives that data and handles process_refund or escalate_to_human, each with separate turn budgets.
- D) Add orchestration-layer code that checks the agent's outcome after each loop termination—if the loop ended without a completed resolution or escalation, programmatically call escalate_to_human with the accumulated conversation context and tool results.

Answer: D

Q93. Agentic Patterns - Tool Error Handling and Recovery Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed?', include what went wrong and what Claude should try next." A billing dispute agent uses lookup_order, which catches all exceptions and returns a tool_result with is_error: true and the message "Tool execution failed". Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls escalate_to_human without trying alternative tools. Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

- A) Implement retry logic with exponential backoff inside each tool implementation so transient errors are resolved transparently within the tool before any failure result is surfaced to Claude in the agent loop.
- B) Remove is_error: true and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior.
- C) Return error-type-specific messages with is_error: true, e.g., "Order not found? try get_customer to search by phone" for data

errors and ?Database timeout (transient)?retry should succeed? for infrastructure errors.

D) Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as ?retry,? ?try_alternative,? or ?escalate,? and appends that recommendation to the tool result.

Answer: C

Q94. Agentic Patterns - MCP Tool Error Communication When implementing your lookup_order MCP tool, the backend sometimes returns errors (e.g., ?Order not found? or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- A) Return a success response with a ?status? field indicating the error type.
- B) Throw an exception from the tool handler so the agent framework can catch and log it.
- C) Return the error message in the tool result content with the isError flag set to true.
- D) Log the error server-side and return an empty result to avoid confusing the model.

Answer: C

Q95. Agentic Patterns - Agent Memory Loss During Multi-Step Verification The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- A) Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
- B) The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- C) The verification tool is clearing the agent's internal state after each successful validation step.
- D) The conversation history isn't being passed in subsequent API requests.

Answer: D

Q96. Agentic Patterns - Managing Large Tool Outputs in Context Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- A) Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions.
- B) Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues.
- C) Proceed with additional lookups without modifying the existing tool output context.
- D) Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details.

Answer: D

Q97. Agentic Patterns - Context Management for Multi-Issue Sessions A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks ?What happened with my refund?? The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- A) Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.
- B) Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
- C) Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- D) Implement sliding window context that retains the most recent 30 turns.

Answer: B

Q98. Agentic Patterns - Guaranteed Compliance for Refund Threshold Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent?this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- A) Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.
- B) Modify the refund tool to return an error with message "Amount exceeds policy limit?please escalate?" when threshold is exceeded.
- C) Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).
- D) Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly."

Answer: A

Q99. Agentic Patterns - Agent Loop Termination Mechanism A customer contacts the agent about a warranty claim on a power drill. Resolving this requires multiple sequential tool calls: `get_customer` to look up their account, `lookup_order` to find the purchase details, and then either `process_refund` or `escalate_to_human` depending on warranty eligibility. You're implementing the agentic loop that orchestrates these steps using the Claude API. What is the primary mechanism your application uses to determine whether to continue the loop or stop?

- A) You manually set the `tool_choice` parameter to "none" after the final expected tool call to force Claude to stop requesting tools.
- B) You check whether Claude's response contains a text content block?if text is present, the agent has produced its final answer and the loop should exit.
- C) You track the number of tool calls made and exit the loop once a preconfigured maximum is reached.
- D) You check the `stop_reason` field in each API response?the loop continues while it equals "tool_use" and exits when it changes to "end_turn" or another terminal value.

Answer: D

Q100. Agentic Patterns - Distinguishing Retriable vs Non-Retriable Tool Errors Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable?", "Connection timeout?") that are transient (~5% of calls), and business errors ("Order exceeds 30-day return window?", "Item already refunded?") that are permanent (~12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude. What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- A) Return structured error responses with `retriable: false` for business errors and a customer-friendly explanation for Claude to use.
- B) Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- C) Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.
- D) Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.

Answer: A

Q101. Agentic Patterns - Tool Selection Accuracy with Semantically Overlapping Tools After expanding the agent's MCP tools with delivery-specific capabilities (`check_delivery_status`, `contact_driver`, `issue_credit`, `apply_promo_code`, `update_delivery_address`, `reschedule_delivery`), the total tool count has grown from 4 to 10. Your evaluation suite shows tool selection accuracy has dropped from 88% to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools?calling `issue_credit` when `process_refund` was correct, and calling `check_delivery_status` when `lookup_order` already returns the needed data. Which approach structurally eliminates the semantic overlap identified in the logs as the error source?

- A) Split the tools across two sub-agents?a "financial resolution" agent with `process_refund`, `issue_credit`, and `apply_promo_code`, and a "delivery operations" agent with the remaining delivery tools?with a coordinator routing between them.
- B) Enable the tool search tool with `defer_loading` on the six new tools, keeping the original four always loaded, so the agent dynamically discovers specialized tools only when needed.
- C) Consolidate semantically overlapping tools?merge `issue_credit` and `process_refund` into a single `resolve_compensation` tool with an `action` parameter, and fold `check_delivery_status` into `lookup_order` with an optional `include_tracking` flag.

D) Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool pair, such as showing when `issue_credit` applies versus when `process_refund` is appropriate.

Answer: C

Q102. Agentic Patterns - Handling Frustrated Customer Requesting Escalation A customer sends: "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools to investigate their account. What should the agent do?

- A) Immediately call `escalate_to_human` with the conversation history.
- B) Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.
- C) First call `get_customer` and `lookup_order` to gather account context, then escalate to a human agent.
- D) Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.

Answer: A

Q103. Agentic Patterns - Escalation Logic Implementation You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

- A) Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- B) Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.
- C) Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- D) Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

Answer: C

Q104. Agentic Patterns - Resuming Sessions with Stale Tool Results A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

- A) Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.
- B) Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.
- C) Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.
- D) Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.

Answer: A

Q105. Agentic Patterns - Context for Human Escalation After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call `escalate_to_human`, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- A) A structured summary: customer ID, root cause, refund amount, and recommended action.
- B) Your diagnosis and the refund amount only.
- C) The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

D) The complete conversation transcript with all tool results.

Answer: A

Q106. Agentic Patterns - Mid-Process Escalation with Context Preservation Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- A) Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.
- B) Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.
- C) Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- D) Call `escalate_to_human` passing only the customer's original message.

Answer: B

Q107. Agentic Patterns - Multi-MCP Server Context Exhaustion Your productivity agent connects to three MCP servers: an issue tracker (`search_issues`, `get_issue`, `create_comment`), a documentation wiki (`search_docs`, `get_page`, `list_spaces`), and a database explorer (`run_query`, `get_schema`, `list_databases`). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations. What architectural change best leverages MCP capabilities to address these issues?

- A) Add a `prepare_investigation` tool to each server that accepts a natural language question and returns relevant content summaries.
- B) Add an orchestrator that routes questions to a single server based on keywords.
- C) Expose each server's content catalog as MCP resources: issue summaries, documentation hierarchy, database schemas.
- D) Consolidate all three servers into a unified MCP server with cross-referencing capabilities.

Answer: C

Q108. Claude Code - Forking Sessions for Parallel Exploration Your agent has analyzed a complex service module reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- A) Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- B) Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.
- C) Export the analysis session's key findings to a file, then create two new sessions that reference this file.
- D) Start two fresh sessions, having each re-read the relevant source files before beginning.

Answer: B

Q109. Claude Code - Exploring Two Refactoring Approaches An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in place. Today, they want to explore both approaches in depth having the agent propose specific code changes for each before deciding which to implement. What's the most effective way to structure this exploration?

- A) Use `fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- B) Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- C) Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.
- D) Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.

Answer: A

Q110. Agentic Patterns - MCP Server Tool Access You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to 'create a branch for JIRA-23 and add documentation links to the ticket,' how does the agent access tools across these servers?

- A) You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- B) The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- C) Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- D) The agent automatically selects the most relevant server based on the request and loads only that server's tools.

Answer: C

Q111. Claude Code - Resuming Interrupted Subagent Session An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- A) Launch a fresh subagent and include the prior transcript in the initial prompt for context.
- B) Launch a fresh subagent with a summary of prior findings.
- C) Resume the subagent from its previous transcript and inform it about the renamed functions.
- D) Resume the subagent from its previous transcript without mentioning the changes; the architecture understanding remains valid.

Answer: C

Q112. Claude Code - Task Decomposition for Intermittent API Errors A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers. What task decomposition approach would be most effective?

- A) Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading.
- B) Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates.
- C) Define a fixed sequence of investigation steps upfront: grep for error patterns, then read error handlers, then check database queries, then examine middleware; executing each step regardless of intermediate findings.
- D) Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges.

Answer: D

Q113. Claude Code - Finding Import Dependencies in Monorepo An engineer asks the agent to find all files in the monorepo that import the @company/auth package to understand how authentication is used across services. Which built-in tool is most appropriate for this task?

- A) Read, starting with package.json files to trace dependency declarations.
- B) Bash, to execute find --type d --name "*auth*" and explore matching directories.
- C) Grep, to search for the import statement pattern across file contents.
- D) Glob, to find files with ?auth? in their filename or path.

Answer: C

Q114. Claude Code - Prioritizing Test Coverage for Legacy Codebase An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize. How should the agent decompose this open-ended task?

- A) Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.
- B) Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.
- C) Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless

of code complexity or business importance.

D) Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning.

Answer: A

Q115. Claude Code - Context Degradation During Large Investigation An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate?it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows. What's the most effective approach to complete this investigation?

A) Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.

B) Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

C) Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.

D) Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding.

Answer: D

Q116. Claude Code - Edit Tool Failure with Non-Unique old_string Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its old_string parameter cannot find unique text to match?the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

A) Use Read to load the file, add the function at the appropriate location, then Write the updated file.

B) Use Bash to append the function definition to the end of the file using heredoc syntax.

C) Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness.

D) Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text.

Answer: A

Q117. Claude Code - Multi-Phase Workflow Benefit An engineer submits two requests: Request A: "Rename the getUserData function to fetchUserProfile everywhere it's used." Request B: "Improve error handling throughout the data processing module?add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data." For which request does specifying an explicit multi-phase workflow (such as analyze ? propose ? implement with review) most improve outcome quality?

A) Both requests benefit equally.

B) Request A, the function rename task.

C) Neither request benefits significantly.

D) Request B, the error handling task.

Answer: D

Q118. Claude Code - Exploring Authentication Architecture in Large Codebase An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude's built-in tools and context limits?

A) Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.

B) Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.

C) Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.

D) Read all files containing ?auth?, ?login?, ?permission?, or ?token? in their content or filename.

Answer: C

Q119. Agentic Patterns - Code Review Task Decomposition Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR. Which task decomposition pattern is most appropriate for this workflow?

A) Prompt chaining?break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

B) Single comprehensive prompt?include all instructions in one prompt and let the model handle all three aspects simultaneously.

C) Routing?classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.

D) Orchestrator-workers?have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

Answer: A

Q120. Claude Code - Integrating Jira Ticket Data Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews?checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copy-pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly. What?s the most effective approach?

A) Build a custom MCP server wrapping Jira?s API with tools designed specifically for this team?s code review workflow.

B) Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

C) Use the Bash tool with curl to call Jira?s REST API, including authentication headers and parsing JSON responses inline.

D) Export Jira tickets to markdown files in the repository that the agent accesses using the Read tool.

Answer: B

Q121. Claude Code - MCP Refactoring Tools Not Being Used After adding an MCP server with specialized code refactoring tools (extract_function, rename_variable, inline_function), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like ?extract_function: Extracts a function from code.? What?s the most effective way to improve adoption of the MCP refactoring tools?

A) Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.

B) Remove the Write tool from the agent?s configuration for refactoring sessions so it must use the MCP tools for code modifications.

C) Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.

D) Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.

Answer: D

Q122. Structured Data Extraction - Reducing False Extractions with Nullable Fields Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article?for example, outputting ?500? for attendee_count when the source contains no attendance information. What?s the most effective way to reduce these false extractions?

A) Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.

B) Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

C) Add prompt instructions to return null for any field where information is not directly stated in the source.

D) Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.

Answer: C

Q123. Structured Data Extraction - Recovering from Pydantic Validation Failures Monitoring shows 12% of extractions fail Pydantic validation with specific errors like "expected float for quantity, got 2 to 3?". Retrying these requests without modification produces identical failures. What's the most effective approach to recover from these validation failures?

- A) Pre-process source documents to standardize problematic formats before sending them for extraction.
- B) Send a follow-up request including the validation error, asking the model to correct its output.
- C) Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation.
- D) Set temperature to 0 to eliminate output variability and ensure consistent formatting.

Answer: B

Q124. Structured Data Extraction - Reliable JSON Schema Compliance Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON. What approach provides the most reliable schema compliance?

- A) Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.
- B) Define a tool with your target schema as input parameters and have Claude call it with the extracted data.
- C) Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.
- D) Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

Answer: B

Q125. Structured Data Extraction - Guaranteed Tool Use for Document Classification The extraction pipeline receives documents of varying types—some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own JSON schema tailored to the document type. During testing, you observe that with tool_choice: "auto", Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- A) Add a preliminary classification call, then make a second call with tool_choice forced to the identified extraction tool.
- B) Consolidate all document types into a single unified-schema extraction tool and force that tool.
- C) Keep tool_choice: "auto" with system prompt instructions requiring tool use.
- D) Set tool_choice: "any" with all extraction tools defined.

Answer: D

Q126. Structured Data Extraction - Handling Inconsistent Formatting in Menus Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting—prices as "\$12" vs "\$12.00", dietary info as icons vs text. What's the most reliable approach?

- A) Define a strict output schema and include format normalization rules in your prompt.
- B) Use separate extraction calls for each field to ensure consistent handling of each type.
- C) Extract data as-is and normalize formats in post-processing code after Claude returns.
- D) Request multiple extraction attempts per document and select the most common format.

Answer: C

Q127. Structured Data Extraction - Message Batches API Batching Strategy Documents arrive continuously throughout business hours and need structured data extracted. To reduce costs, you want to use the Message Batches API (50% discount, up-to-24-hour processing window). Your SLA specifies that extraction results must be available within 30 hours of document arrival with 99.9% reliability. Which batching strategy is most appropriate?

- A) Submit batches every 4 hours containing documents from that window.
- B) Submit a single batch at end of day containing all documents from that day.
- C) Use the real-time API for all documents instead of batch processing.
- D) Submit batches every 6 hours containing documents from that window.

Answer: D

Q128. Structured Data Extraction - Ensuring Metadata Extraction Before Enrichment Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like `?extract the metadata and tell me how cited it is,` Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- A) Set `tool_choice` to `?auto?` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- B) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- C) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- D) Set `tool_choice` to `?any?` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.

Answer: B

Q129. Structured Data Extraction - Schema Design to Prevent Hallucinated Values Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated `?weight: 2.3 kg?` in the extraction output. What schema design change most effectively addresses this hallucination behavior?

- A) Add explicit instructions to the prompt stating `?only extract information explicitly stated in the document; use placeholder text for missing values.?`
- B) Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text.
- C) Change fields that may not exist in source documents from required to optional, allowing the model to omit them.
- D) Add a `?confidence?` field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions.

Answer: C

Q130. Structured Data Extraction - Human Review Routing Strategy Your extraction pipeline validates outputs against JSON schemas, but you need to implement human review given limited reviewer capacity (they can handle approximately 5% of total extraction volume). What's the most effective basis for selecting which extractions to route for human review?

- A) Route extractions where the model indicates low confidence or where source documents contain ambiguous or contradictory information.
- B) Route extractions for review only when downstream systems report data quality issues or processing failures.
- C) Randomly sample 5% of extractions for review.
- D) Route extractions containing specific high-priority entity types (e.g., financial figures, dates) for human review, regardless of extraction confidence.

Answer: A

Q131. Structured Data Extraction - Handling Evolving Enum Values Your extraction uses tool use with a JSON schema where `property_type` is defined as an enum: `["house", "apartment", "condo", "townhouse"]`. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types like `studio`, `loft`, `duplex`, `mobile home`, `tiny house`, `converted warehouse` and new types continue appearing regularly. What's the most effective long-term solution?

- A) Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.
- B) Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.
- C) Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- D) Add an `other` value to your enum with a separate `property_type_detail` string field for specifics when `other` is selected.

Answer: D

Q132. Structured Data Extraction - Schema Modification for Ambiguous Reviews The system processes product reviews using tool use with a defined schema: `rating` (integer 1-5), `pros` (string array), `cons` (string array), and `overall_sentiment` (enum: `positive`, `negative`, `mixed`). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like `Great product!`, Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like `Well that was interesting`, Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

- A) Allow empty arrays for pros/cons as valid output, and add `unclear` to the sentiment enum.
- B) Make pros and cons optional fields, and add `neutral` and `unclear` to the sentiment enum.
- C) Allow null values for pros/cons, and add `unclear` to the sentiment enum.
- D) Add an `extraction_confidence` field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

Answer: A

Q133. Structured Data Extraction - Context Window and Tool Definition Token Usage Your extraction system uses tool use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens. What is the most likely cause?

- A) Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.
- B) Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.
- C) The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.
- D) Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.

Answer: B

Q134. Structured Data Extraction - Pipeline Architecture for Mixed Urgency Documents Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- A) Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- B) Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- C) Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.
- D) Submit all documents to the Batch API with `custom_ids` for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

Answer: C

Q135. Structured Data Extraction - Ensuring Valid JSON Output Your extraction pipeline occasionally receives responses that don't parse as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it. What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

- A) Implement a retry loop that catches JSON parse errors and re-prompts Claude with the error details, asking it to correct the malformed output.
- B) Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.
- C) Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.
- D) Define a tool with a JSON schema specifying the expected structure, using tool use to constrain Claude's output to schema-compliant JSON.

Answer: D

Q136. Structured Data Extraction - Validating Before Automating High-Confidence Extractions Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence >90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- A) Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- B) Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- C) Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- D) Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.

Answer: C

Q137. Claude Code - Debugging Unfamiliar Module with Clear Stack Trace A critical bug is affecting production users. Error logs show exceptions in the OrderProcessing module with a clear stack trace pointing to a specific function. You haven't worked with this module before. What's the most effective approach?

- A) Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically.
- B) Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes.
- C) Enter plan mode to explore the module's architecture and dependencies before attempting any fixes.
- D) Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause.

Answer: D

Q138. Claude Code - Following Established Patterns for New Module You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging. You've identified three existing modules that exemplify these patterns: db_utils.py, error_handlers.py, and audit_logger.py. This is a one-off integration task; these patterns are well-documented in your team wiki and don't need additional project-level documentation. What's the most effective approach?

- A) Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow.
- B) Use @ references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow.
- C) Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically.
- D) Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module.

Answer: B

Q139. Claude Code - Restructuring Configuration Mechanisms Your team has three requirements for Claude Code's behavior in your project: 1. Claude must never modify files in the `db/migrations/` directory 2. Claude should prefer your custom logging module over `console.log` 3. All TypeScript files must be auto-formatted with Prettier after every edit All three are currently written as instructions in your project's `CLAUDE.md`. During a complex refactoring session, a developer discovers that Claude edited a migration file, violating requirement #1. How should you restructure these requirements across Claude Code's configuration mechanisms?

- A) Add `!db/migrations/**` to `permissions.deny` in the project settings, keep the logging preference in `CLAUDE.md`, and add a `PostToolUse` hook on the Edit tool that runs Prettier on changed TypeScript files.
- B) Rewrite all three requirements in `CLAUDE.md` using stronger directive language and add few-shot examples that demonstrate Claude refusing to edit migration files and running Prettier after edits.
- C) Move all three requirements into `.claude/rules/` as path-scoped rules: one targeting `db/migrations/**` that forbids editing those files, and others targeting `**/*.ts` for the logging convention and formatting.
- D) Configure hooks for all three: a `PreToolUse` hook script that blocks Edit calls targeting `db/migrations/`, a `PreToolUse` hook script that adds logging convention context before edits, and a `PostToolUse` hook that runs Prettier after TypeScript edits.

Answer: A

Q140. Claude Code - Simple Conditional Addition Approach You need to add a date validation check ensuring event dates are in the future. This requires adding a conditional statement to one existing function in a single file. What is the most appropriate approach?

- A) Enter plan mode to analyze how the validation might impact other parts of the reservation flow.
- B) Enter plan mode first to create a detailed implementation strategy before making the change.
- C) Use direct execution to make the change.
- D) Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic.

Answer: C

Q141. Claude Code - Inconsistent Error Handling Convention Adherence You've documented API error handling conventions in a `CLAUDE.md` file at your project root, specifying that endpoint handlers should use a custom `ApiError` class. After several sessions, you notice Claude Code sometimes follows these conventions and sometimes uses generic `try/catch` blocks with string messages. The inconsistency appears random across different coding sessions. What's the most efficient first diagnostic step?

- A) Run `/memory` to check which memory files are loaded and verify your `CLAUDE.md` is included.
- B) Search for conflicting instructions in `~/.claude/CLAUDE.md` or `~/.claude/rules/` that might override your project conventions.
- C) Create path-specific rules in `.claude/rules/handlers.md` with YAML frontmatter scoping the error handling instructions to your API handler files.
- D) Add more detailed code examples to your `CLAUDE.md` showing the exact `ApiError` usage pattern for different endpoint types.

Answer: B

Q142. Claude Code - MCP Server Configuration Scopes Your team is configuring MCP servers in Claude Code. You want to add a shared venue lookup server that all team members should have access to, and you personally want to add an experimental music playlist server that only you are testing. Which configuration approach correctly applies MCP server scopes?

- A) Add both servers to your local `~/.claude.json`
- B) Add venue server to `~/.claude.json` and playlist server to `.mcp.json`
- C) Add both servers to the project-level `.mcp.json` file
- D) Add venue server to `.mcp.json` and playlist server to `~/.claude.json`

Answer: D

Q143. Claude Code - Iterating on Interacting Formatting Issues You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact?changing column widths affects how dates render, and page breaks depend on content height. What's the most effective approach for iterating toward a working solution?

- A) Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.
- B) Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.
- C) Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks?testing after each change.
- D) Start fresh with a detailed prompt specifying all formatting requirements upfront.

Answer: C

Q144. Claude Code - Real-Time Updates Architecture Decision You're tasked with adding real-time updates to the application. This could be implemented using WebSockets, Server-Sent Events, or polling, each with different complexity, browser support, and infrastructure requirements. What's the most effective way to begin this task?

- A) Enter plan mode to explore the architecture, evaluate trade-offs, and present options for team approval before implementing.
- B) Use direct execution with a prompt asking Claude to analyze all approaches and implement the one it determines is best.
- C) Use direct execution to implement polling first, then evaluate whether to upgrade to WebSockets later.
- D) Start direct execution with WebSockets, then refactor if infrastructure issues arise.

Answer: A

Q145. Claude Code - MCP Prompts Accessibility Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as `deploy_checklist` and `incident_response`) in addition to tools. How do these MCP prompts become accessible within Claude Code?

- A) They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.
- B) They appear as slash commands (e.g., `/mcp_servername_deploy_checklist`) that you can invoke, with arguments passed after the command name.
- C) They are surfaced as @-mentionable resources alongside files, fetched and attached to your message when referenced.
- D) They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

Answer: B

Q146. Claude Code - Iterating on Data Migration Script with Null Handling You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields. What's the most effective way to iterate toward a working solution?

- A) Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.
- B) Provide a test case with example input containing null values and the expected output, then ask Claude to fix it.
- C) Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.
- D) Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.

Answer: B

Q147. Claude Code - Monorepo CLAUDE.md Standards Organization Your monorepo contains shared coding standards in `/docs/standards/`: `security-rules.md` (for services handling user data), `testing-patterns.md` (for all packages), and `api-conventions.md` (for API-facing services). Your 15 packages are organized by feature domain (`/packages/auth/`, `/packages/billing/`, `/packages/notifications/`, etc.) without naming conventions indicating which handle user data or expose APIs. Package maintainers are expected to configure their own local development settings, as they understand their package's domain requirements. Currently, all package CLAUDE.md files duplicate all three standards, applying irrelevant guidance. What's the most effective approach?

- A) Use `@imports` in each package's CLAUDE.md to reference only the specific standard files relevant to that package, based on the maintainer's domain knowledge.
- B) Create a `shared-standards.md` that uses `@imports` to combine all three standards, then have each package's CLAUDE.md import that combined file.
- C) Put all standards in the root CLAUDE.md with override instructions like `?ignore security-rules.md when working in packages that don't handle user data.`
- D) Create `.claude/rules/` files for each standard with YAML frontmatter paths listing every package directory where that standard should apply.

Answer: A

Q148. Claude Code - Starting Iterative Workflow for Caching Implementation You're implementing a caching layer for API responses to speed up the `/products` endpoint. You have a rough idea of Redis with a 5-minute TTL but you're new to production caching and aren't sure what other considerations a robust implementation requires. What's the most effective way to start your iterative workflow?

- A) Write a specification with your known requirements and `?TBD?` markers for uncertain areas, having Claude propose solutions for each TBD as it implements.
- B) Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes.
- C) Start with a minimal request: `?Add Redis caching to /products with 5-minute TTL.` Add features and fix issues through follow-up prompts as problems surface during testing.
- D) Use plan mode to analyze the current `/products` endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.

Answer: B

Q149. Claude Code - Shared Skill File Location for Team Workflow Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing `/migrate-component`. The workflow should stay in sync as the team iterates on it. Where should you place the skill file?

- A) In `.claude/skills/migrate-component/SKILL.md` at the project root, committed to version control.
- B) In the project's `.claude/settings.json` using a `skillOverrides` entry to register and define the workflow.
- C) As a detailed instruction block in the project's root CLAUDE.md file.
- D) In `~/.claude/skills/migrate-component/SKILL.md` on each developer's machine.

Answer: A

Q150. Claude Code - Code Review Checklist Workflow Your team wants Claude to follow a detailed code review checklist (8 items covering API changes, test coverage, documentation, security, etc.) when reviewing pull requests. The team also uses Claude extensively for other tasks: writing new features, debugging production issues, and generating documentation. Currently, developers paste the checklist at the start of each review session. Which approach best addresses this workflow need?

- A) Create a dedicated review subagent with the checklist embedded in its configuration.
- B) Create a `/review slash` command containing the checklist, invoked when starting reviews.
- C) Add the checklist to the project's CLAUDE.md file under a `?Code Review?` section.
- D) Configure plan mode as the default for code review sessions.

Answer: B

Q151. Claude Code - Path-Scoped Rules for Infrastructure Repo Your infrastructure-as-code repository includes Terraform modules (/terraform/), Kubernetes manifests (/kubernetes/), and CI/CD pipeline scripts (/pipelines/). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens. What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

- A) Keep the root CLAUDE.md and use @path/to/import syntax to modularly include tool-specific guidance files from separate documents.
- B) Split content into subdirectory CLAUDE.md files (/terraform/CLAUDE.md, /kubernetes/CLAUDE.md), so Claude loads directory-specific guidance.
- C) Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., ## Terraform Conventions?), improving organization and readability.
- D) Create files in .claude/rules/ with YAML frontmatter path scoping (e.g., paths: ["terraform/**/*"]), loading rules only when editing matching files.

Answer: D

Q152. Agentic Patterns - Addressing Depth Gaps in Research Pipeline Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps—for instance, noting the retrieved sources discuss API authentication but lack details on token refresh patterns—but under the current strict pipeline, this insight isn't actionable since search has already completed. What's the most effective architectural change?

- A) Add a research planning agent before the search phase that decomposes topics into specific sub-questions.
- B) Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.
- C) Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.
- D) Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

Answer: B

Q153. Agentic Patterns - Coordinator Not Spawning Subagents The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

- A) The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.
- B) The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.
- C) The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.
- D) Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.

Answer: C

Q154. Agentic Patterns - Improving Subagent Adaptability The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports 'insufficient results' rather than trying alternative approaches when pre-specified searches fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- A) Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.
- B) Implement a topic classification step where the coordinator categorizes requests as 'well-defined' or 'exploratory' and uses different instruction styles for each category.
- C) Remove procedural details entirely, delegating with simple goals like 'research X thoroughly' and relying on the subagent's general capabilities.
- D) Add explicit fallback directives to the detailed instructions: 'If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure.'

Answer: A

Q155. Agentic Patterns - Improving Tool Reliability with Ambiguous Instructions The document analysis agent has a single `analyze_document` tool that takes a document and a free-text instruction parameter. During evaluation, requests like 'extract the key financial metrics' often return narrative summaries, while 'summarize the methodology' sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

- A) Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats.
- B) Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent.
- C) Keep the single tool but add an `analysis_type` enum parameter requiring explicit selection between extraction, summarization, and verification modes.
- D) Split the generic tool into purpose-specific tools: `extract_data_points`, `summarize_content`, `verify_claim_against_source` each with defined input/output contracts.

Answer: D

Q156. Agentic Patterns - State Management for Crashed Multi-Agent Pipeline Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

- A) Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.
- B) Have each agent maintain its own persistent state file and reload it independently at the start of each session.
- C) Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.
- D) Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

Answer: D

Q157. Agentic Patterns - Passing Information Between Subagents After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

- A) Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results.
- B) Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks.
- C) Include the complete findings from both subagents directly in the synthesis subagent's prompt.

D) Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator.

Answer: C

Q158. Agentic Patterns - Improving Research Completeness with Feedback Loop The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A) Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

B) Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

C) Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

D) Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.

Answer: D

Q159. Agentic Patterns - Information Flow Between Subagents The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information typically flow between these two specialized subagents?

A) The coordinator receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

B) Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

C) The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.

D) The web search agent directly invokes the document analysis agent, passing the discovered sources as parameters.

Answer: A

Q160. Agentic Patterns - Improving Briefing Quality with Mixed Content Types A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

A) Update the synthesis agent to render each content type appropriately?financial data as tables, news as prose.

B) Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation.

C) Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence.

D) Standardize all subagent outputs to prose summaries with inline citations.

Answer: A

Q161. Agentic Patterns - Synthesis Agent Cannot Complete Task After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

A) The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

B) The subagents need to share a single API connection to enable automatic context sharing between invocations.

C) The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

D) The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.

Answer: C

Q162. Agentic Patterns - Source Attribution in Multi-Agent Pipeline In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

A) Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

B) Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

C) Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

D) Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

Answer: B

Q163. Agentic Patterns - Context-Passing Strategy for Report Generation After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

A) Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

B) Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.

C) Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.

D) Pass the full accumulated context from all prior agents.

Answer: C

Q164. Agentic Patterns - Preserving Source Attribution Through Summarization The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations?the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

A) Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

B) Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.

C) Have the report generator query the web search agent to re-locate sources for claims in the final report.

D) Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.

Answer: B

Q165. Agentic Patterns - Temporal Data Interpretation in Synthesis When researching ?renewable energy adoption,? the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2021: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

A) Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

B) Require subagents to include publication or data collection dates in their structured outputs.

C) Configure the web search agent to only return results from the past 6 months.

D) Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical

appendix.

Answer: B

Q166. Agentic Patterns - Poor Tool Selection with Shared Tool Access You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization?the synthesis agent attempts web searches, and the report generator tries to analyze documents. What is the primary cause of this poor tool selection behavior?

- A) The tool definitions consume too much context window space, leaving insufficient room for task content.
- B) Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.
- C) The agents' role descriptions in their system prompts conflict with having access to tools outside that role.
- D) The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.

Answer: B

Q167. Agentic Patterns - Automated Code Review Missing Bugs Your automated code review is missing genuine bugs in pull requests. Investigation reveals that your review prompt includes the instruction: "Only flag critical issues that would definitely cause production failures. Ignore minor concerns and anything you're uncertain about." Developers confirm that some missed bugs are real logic errors the model investigated but chose not to report. The team requires that review output remain structured (each finding tagged with metadata) and actionable. Which prompt change both removes the cause of the suppressed findings and preserves structured, tagged output for downstream filtering?

- A) Enable extended thinking and instruct the model to reason step-by-step about each code change before producing its review.
- B) Add a second review pass that re-reads the diff using the same prompt, looking for anything the first pass may have missed.
- C) Remove all severity-related instructions from the prompt and let the model use its default judgment about what to report.
- D) Instruct the model to report all findings with a confidence level and severity tag, deferring filtering to a downstream step.

Answer: D

Q168. Prompt Caching - Cache Misses in Message Batches API Your pipeline reviews approximately 200 database migration scripts daily using the Message Batches API. Each batch request includes a shared 8,000-token system prompt containing migration review guidelines and schema documentation, followed by the individual migration script. You've added cache_control breakpoints on the shared system prompt in every request, but monitoring shows cache hit rates of only 32%, with cache misses concentrated on requests processed later in the batch window. Which change will address the root cause of these cache misses without adding sequential processing latency?

- A) Add pre-warming requests with max_tokens: 0 at the beginning of each batch to seed the cache before the review requests execute.
- B) Split the 200 requests into 10 sequential batches of 20, submitting each only after the previous completes, to keep requests temporally close enough for cache reuse.
- C) Use the extended 1-hour cache TTL instead of the default 5-minute TTL on your cache breakpoints.
- D) Move the cache_control breakpoints from the system prompt to the individual migration script content to cache similar code patterns across reviews.

Answer: C

Q169. Claude Code - Reducing Noise in Automated Code Review Pipeline The automated review pipeline generates an average of 18 findings per PR. Developer feedback reveals three categories of unwanted noise: (1) style and formatting issues already enforced by your linter in CI, (2) findings on auto-generated template code under src/gen/, and (3) rendering helper patterns that are intentional project conventions but get flagged because they resemble common anti-patterns. Only about 4 findings per PR are genuine logic bugs. What is the most effective way to reduce this noise while preserving detection of real issues?

- A) Create a REVIEW.md at the repository root with skip rules for CI-enforced checks and generated files, and a verification requirement that rendering-related findings must cite a specific line demonstrating incorrect behavior.
- B) Add the prompt parameter to the GitHub Action configuration with instructions to suppress lint-duplicate findings, ignore

generated template code, and apply stricter evidence requirements for rendering-related issues.

C) Add detailed explanations to the project's CLAUDE.md describing which patterns are intentional, that linting is handled separately by CI, and that the src/gen/ directory contains auto-generated template code.

D) Configure separate GitHub Action workflow files for each code area—one for generated code with findings suppressed, one for rendering code with custom review instructions, and a general workflow for everything else.

Answer: C

Q170. Claude Code - Enabling Repository File Reading in Piped Code Review Your pipeline runs: `prompts="You are a code reviewer." prompt="$prompt Analyze the provided diff" prompt="$prompt for bugs, security issues," prompt="$prompt and style violations." claude -p \ --dangerously-skip-permissions \ --system-prompt "$prompt" \ < diff.txt` The reviews complete and return feedback, but Claude only comments on the piped diff text?it never reads surrounding files in the checked-out repository to understand broader context, even when the diff modifies a function called by many other modules. Which change to the invocation will cause Claude to read related files in the repository while still applying your custom review instructions?

A) Remove --system-prompt entirely and place the review instructions in a CLAUDE.md file at the repo root, since --system-prompt is incompatible with tool use under -p.

B) Stop piping the diff via stdin and instead embed the diff contents inside the prompt string, so Claude Code treats the invocation as an agentic session rather than a stream-processing one.

C) Replace --system-prompt with --append-system-prompt so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools.

D) Keep --system-prompt and add --allowedTools "Read,Glob,Grep" so that the non-interactive -p mode permits file system tools that it otherwise disables.

Answer: C

Q171. Claude Code - Cross-File Bug Detection in Code Review Your pipeline reviews every PR using a single API call with a static prompt containing the diff and full text of each changed file?unchanged files are not included. Reviews are posted asynchronously and don't block PR creation. Developers report that reviews consistently miss bugs involving cross-file interactions?for example, a PR renames a function's parameters but the review doesn't flag callers in unchanged files that still use the old argument order. Evaluation shows cross-file bugs account for 35% of production incidents from reviewed PRs. What is the most effective change to your review design?

A) Redesign the review as a turn-limited agentic task where the model can read files and search the codebase via tools, following references to verify cross-file findings.

B) Run parallel review passes per changed file with direct dependents included in each pass, then aggregate and deduplicate findings using a final summarization call.

C) Use static analysis to build a dependency graph of changed code, then expand the prompt to include all files within two dependency hops of any changed file.

D) Add chain-of-thought instructions asking the model to list all external references in the diff, then reason step-by-step about how each change might affect callers in other files.

Answer: A

Q172. Claude Code - Capping Iteration Count and Cost in CI Reviews During initial testing of the automated review pipeline, you notice that reviews on large PRs (50+ changed files) sometimes take over 20 minutes and cost \$8-12 per run due to extensive agentic loops?Claude reads files, runs analysis tools, and iterates many times. Your team needs each invocation to abort once it reaches a fixed iteration count and a fixed dollar amount, enforced by Claude Code itself rather than the surrounding job runner. Which configuration change directly enforces both of those per-invocation caps?

A) Add --max-turns 30 --max-budget-usd 2.00 to the claude -p invocation to cap iterations and spend.

B) Set timeout-minutes: 5 on the GitHub Actions job step and monitor per-run costs via the Anthropic Console usage dashboard.

C) Set --permission-mode dontask to auto-deny any tool permission requests not in the explicitly allowed set.

D) Switch the --model flag to a smaller, cheaper model so each iteration uses fewer tokens and lower per-call cost.

Answer: A

Q173. Claude Code - Same-Session Review Missing Bugs A developer uses Claude Code to refactor a function during their development session. Before committing, they ask the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed. What best explains this discrepancy?

- A) The CI environment has access to the full codebase context while the local session only sees the current file.
- B) The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general.
- C) Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions.
- D) The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis.

Answer: C

Q174. Claude API - Handling Truncated JSON from Tool Use in Large PRs Your automated review calls the Claude API for each PR, using `tool_use` with a `report_findings` tool that returns a JSON array of finding objects (each with `file_path`, `line_number`, `severity`, `category`, and `description`). During testing on a large PR touching 30+ files, the response hits the `max_tokens` limit and the output is truncated mid-JSON, causing your pipeline's parser to fail. What is the most effective way to handle this?

- A) Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings.
- B) Switch from `tool_use` to prompting Claude to return findings as a markdown list.
- C) Increase `max_tokens` to the model's maximum and instruct Claude to keep finding descriptions under 50 words each.
- D) Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.

Answer: D

Q175. Claude Code - Reducing False Positives from Project Conventions The automated review consistently flags patterns your team uses intentionally: force-unwrapping optionals in test files, using large coordinator classes that follow your established architecture, and importing internally-maintained modules marked as deprecated in the public SDK. Developers are dismissing roughly 30% of all findings as project-specific false positives. Which approach prevents the model from generating these findings in the first place by supplying the project's conventions as persistent context on every review?

- A) Configure the review to analyze only the changed lines in the diff without surrounding file context, reducing the amount of code the model evaluates per review.
- B) Document the team's accepted patterns and intentional conventions in the project's `CLAUDE.md` file so the model receives this context during every review.
- C) Have developers add inline suppress-comments at flagged lines and preprocess diffs to exclude suppressed lines before sending code to the model.
- D) Build post-processing keyword filters that suppress findings containing terms like `?force unwrap?`, `?large class?`, or `?deprecated import?` before results reach developers.

Answer: B

Q176. Claude API - Reducing Cost for Release Notes Generation with Message Batches Your pipeline includes a release notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes aren't needed until the following morning (results have ~12 hours of acceptable latency). Your team needs to reduce per-token API cost for this step while keeping the same model and prompts (no change to model tier or output quality). Which approach satisfies all of these constraints?

- A) Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, since fewer requests always reduces total token cost.
- B) Submit the 200 requests to the Message Batches API with unique `custom_ids` and retrieve results once the batch ends, which applies a 50% discount to all input and output tokens.
- C) Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token

rates.

D) Issue the 200 Messages API requests in parallel using concurrent connections, since concurrency lowers the per-token price charged by the API.

Answer: B

Q177. Claude Code - High Precision but Low Recall in Automated Review After deploying the automated review, you notice high precision but low recall?real bugs are slipping through undetected. Investigation reveals your review prompt instructs Claude to ?only report high-confidence issues you are certain about? and ?err on the side of not commenting.? Developers appreciate the low noise, but a race condition that caused a production outage was visible in a reviewed PR and went unreported. You need to substantially improve bug detection while keeping false positive rates manageable for your team. What is the most effective approach?

A) Split the review into a finding stage where Claude?s goal is coverage?flagging every potential issue with confidence and severity metadata?and a separate stage that thresholds those findings.

B) Remove the conservative filtering instructions and prompt Claude to report all potential issues, then apply a programmatic filter to deduplicate and suppress categories that historically generate false positives.

C) Add detailed few-shot examples demonstrating bug categories Claude should flag?race conditions, null dereferences, error handling gaps?while keeping the high-confidence filtering instruction to maintain current precision levels.

D) Expand the context window by including related test files, recent git history, and the module?s dependency graph alongside the diff, giving Claude richer signals to assess issue severity.

Answer: A

Q178. Claude Code - Reducing CI Startup Time with Bare Mode Your automated review CI jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals the delay comes from auto-discovery of hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout your monorepo. You need to cut startup time while ensuring reviews still enforce your team?s coding standards, which are documented in your root-level CLAUDE.md file. What is the most effective approach?

A) Replace the default prompt entirely using --system-prompt-file /CLAUDE.md, which bypasses default prompt assembly and loads only your project rules.

B) Run with --bare mode and specify all review criteria directly in the -p prompt argument for each CI invocation, without referencing any external files.

C) Run with --bare mode and pass --append-system-prompt-file /CLAUDE.md to explicitly load your project standards while skipping all auto-discovery.

D) Keep the default initialization and add --exclude-dynamic-system-prompt-sections to reduce per-machine prompt variability and improve prompt cache hit rates across runners.

Answer: C

Q179. Claude Code - Analyzing Dismissal Patterns in Code Review Output You built an LLM-powered code review tool that analyzes pull requests and outputs structured findings. Each finding is a JSON object with file_path, line_number, issue_category (e.g., ?security?, ?style?), and description. Developers can click ?dismiss? on any finding they consider unhelpful?currently 35% of findings get dismissed. You want to analyze these dismissals to understand what your system is getting wrong and improve your prompts accordingly. What change to your output structure would best support this analysis?

A) Add a model_confidence field (0.0-1.0) and filter out findings below a threshold calibrated against historical dismiss rates.

B) Remove the issue_category field and track dismiss rates at the individual finding level only.

C) Add a detected_pattern field recording the code construct (e.g., ?single-letter loop variable?) that triggered each finding.

D) Expand the description field to include more detailed explanations of why each issue matters and how to fix it.

Answer: C

Q180. Claude Code - Reducing Dismissals with Prompt Design Your automated review generates many findings per PR, but developer feedback shows roughly half are dismissed as "not worth addressing." Analysis reveals dismissed findings are often technically accurate but involve minor style preferences or patterns acceptable in your codebase. Before adding infrastructure complexity, what prompt design change would most effectively reduce dismissals while maintaining detection of genuine issues?

- A) Add instructions asking Claude to rate each finding's confidence (1-10) and only include findings rated 8 or higher.
- B) Append instructions to "only report findings you are highly confident are genuine problems."
- C) Add explicit criteria defining which issues to report (bugs, security) versus skip (minor style, local patterns).
- D) Implement a secondary classification model that filters Claude's findings based on predicted developer acceptance.

Answer: C

Q181. Claude Code - Reducing Low-Value Test Generation Your test generation produces unit tests for new code, but reviews show 55% are low-value: trivial assertions that only verify functions don't throw exceptions, tests duplicating existing coverage, or tests ignoring your team's fixture conventions. How do you reduce the rate of low-value tests being generated in the first place?

- A) Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers.
- B) Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing.
- C) Add post-generation coverage analysis that automatically filters out any generated test that doesn't increase line coverage beyond what existing tests provide.
- D) Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions.

Answer: D

Q182. Structured Data Extraction - Forcing Tool Order for Metadata Extraction Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- A) Set `tool_choice` to "auto" and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- B) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- C) Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- D) Set `tool_choice` to "any" so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.

Answer: B

Q183. Structured Data Extraction - Schema Design for Ambiguous Reviews The system processes product reviews using tool use with a defined schema: `rating` (integer 1-5), `pros` (string array), `cons` (string array), and `overall_sentiment` (enum: `positive`, `negative`, `mixed`). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was? Interesting?", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

- A) Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.
- B) Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.
- C) Allow null values for pros/cons, and add "unclear" to the sentiment enum.
- D) Add an `extraction_confidence` field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.

Answer: C

Q184. Claude Code - Debugging Critical Bug in Unfamiliar Module A critical bug is affecting production users. Error logs show exceptions in the `OrderProcessing` module with a clear stack trace pointing to a specific function. You haven't worked with this module before. What's the most effective approach?

- A) Use plan mode to analyze the error in context of the module's design, enumerate potential root causes, and prioritize fixes systematically.
- B) Start with direct execution to gather initial information, then switch to plan mode to design a comprehensive solution before implementing any changes.
- C) Enter plan mode to explore the module's architecture and dependencies before attempting any fixes.
- D) Use direct execution to examine the stack trace, read the relevant code, and implement a fix once you identify the root cause.

Answer: D

Q185. Claude Code - MCP Prompts as Slash Commands Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as `deploy_checklist` and `incident_response`) in addition to tools. How do these MCP prompts become accessible within Claude Code?

- A) They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.
- B) They appear as slash commands (e.g., `/mcp__servername__deploy_checklist`) that you can invoke, with arguments passed after the command name.
- C) They are surfaced as @-mentionable resources alongside files, fetched and attached to your message when referenced.
- D) They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.

Answer: B

Q186. You're implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what users said earlier in the same conversation?for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most likely cause?

- A) The model's context window has been exceeded by the conversation length
- B) The Claude API requires a `session_id` parameter that you haven't configured
- C) Your application isn't including prior messages in the `messages` array
- D) Claude requires a vector database connection to maintain conversation memory

Answer: C

Your application isn't including prior messages in the `messages` array. Claude is stateless?each API call must include the full conversation history in the `messages` array for Claude to remember prior context. If your app only sends the latest message, Claude has no memory of earlier turns.

Q187. Your research assistant helps users analyze academic papers over extended conversations. User testing reveals a recurring issue: after conversations exceed 60K tokens, users ask follow-up questions requiring precise numerical details from papers discussed earlier?sample sizes, exact p-values, specific inclusion criteria. Your current approach summarizes paper discussions after 8 turns to stay within context limits. Users report that responses to these precision-dependent questions are often hedged or inaccurate. What's the most effective architectural change?

- A) Use a separate Claude call with explicit instructions to generate higher-fidelity summaries that preserve all numerical details and statistical values.
- B) Implement retrieval that re-injects relevant paper sections when the user's question suggests they need specific numerical details.
- C) Maintain a structured database of key facts extracted from each paper (sample sizes, statistics, methods) and retrieve relevant entries into context when precision-dependent questions are detected.
- D) Keep source text from methodology and results sections in context permanently, while summarizing only the conversational discussion and interpretation portions.

Answer: C

Maintain a structured database of key facts extracted from each paper (sample sizes, statistics, methods) and retrieve relevant entries into context when precision-dependent questions are detected. This approach specifically addresses the problem of losing precise numerical details during summarization by storing them in a structured format that can be reliably retrieved when needed, without consuming excessive context space.

Q188. Your music discovery assistant should consistently maintain an enthusiastic tone, explain its reasoning for each recommendation, and ask clarifying questions to better understand user preferences. You want this behavior to persist reliably across all user interactions. Where should you define these behavioral guidelines?

- A) Prepended to each user message before sending to the API
- B) In the system prompt
- C) In environmental variables that your application passes to the API client
- D) In the first assistant message, instructing Claude to follow these guidelines going forward

Answer: B

In the system prompt. The system prompt is specifically designed to define persistent behavioral guidelines, tone, and personality traits that should apply across all interactions. It's processed before any user messages and provides consistent instructions for every response.

Q189. Users report that responses feel repetitive across turns?each message begins with phrases like ?Certainly!? or ?I'd be happy to help!? even deep into conversations. You want responses to flow naturally without these repetitive openers. What's the most effective approach?

- A) Lower the temperature parameter to make response openings more deterministic and less variable
- B) Implement post-processing to detect and strip common greeting phrases from response beginnings
- C) Add system prompt instructions specifying phrases to avoid, such as ?Never begin responses with ?Certainly? or similar affirmations?
- D) Append a partial assistant message with a direct response opening that the model will continue from

Answer: D

Append a partial assistant message with a direct response opening that the model will continue from. By prefilling the assistant response with a direct opening, you force the model to continue from that point, eliminating the repetitive opener pattern entirely. This is more reliable than instructions (which can be ignored) or post-processing (which is a workaround).

Q190. An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth?having the agent propose specific code changes for each?before deciding which to implement. What's the most effective way to structure this exploration?

- A) Use fork_session to create two branches from yesterday's analysis, exploring one approach in each fork.
- B) Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- C) Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.
- D) Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Answer: A

Use fork_session to create two branches from yesterday's analysis, exploring one approach in each fork. This preserves the full context from yesterday's analysis in both branches, allows independent exploration without cross-contamination between approaches, and avoids manual context recreation.

Q191. You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search. When a user asks the agent to ?create a branch for JIRA-123 and add documentation links to the ticket,? how does the agent access tools across these servers?

- A) The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- B) You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- C) The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- D) Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.

Answer: D

Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent. MCP servers expose their tools during initialization, and the agent has access to all tools from all connected servers at once, allowing it to use git tools, Jira tools, and documentation tools in a single turn.

Q192. An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- A) Resume the subagent from its previous transcript without mentioning the changes; the architecture understanding remains valid.
- B) Launch a fresh subagent with a summary of prior findings.
- C) Launch a fresh subagent and include the prior transcript in the initial prompt for context.
- D) Resume the subagent from its previous transcript and inform it about the renamed functions.

Answer: D

Resume the subagent from its previous transcript and inform it about the renamed functions. This preserves the 30 minutes of accumulated context and architectural understanding while updating the agent about the specific changes (renamed functions) that could cause confusion if not acknowledged.

Q193. An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate; it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows. What's the most effective approach to complete this investigation?

- A) Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding.
- B) Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.
- C) Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.
- D) Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

Answer: A

Spawn subagents to investigate specific questions while the main agent coordinates findings and preserves high-level understanding. This delegates context-heavy work to subagents with fresh context windows while the main agent maintains the architectural overview, avoiding context exhaustion and enabling parallel investigation.

Q194. Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its old_string parameter cannot find unique text to match ; the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- A) Use Bash to append the function definition to the end of the file using heredoc syntax
- B) Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness
- C) Use Read to load the file, add the function at the appropriate location, then Write the updated file
- D) Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text

Answer: C

Use Read to load the file, add the function at the appropriate location, then Write the updated file. When the Edit tool's string matching fails due to repetitive patterns, the most reliable fallback is to read the entire file, construct the complete updated content with the insertion at the correct line, and write the whole file back.

Q195. Your agent has analyzed a complex service module; reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- A) Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- B) Start two fresh sessions, having each re-read the relevant source files before beginning.
- C) Resume analysis session with fork_session enabled, creating a separate branch for each testing strategy.
- D) Export the analysis session's key findings to a file, then create two new sessions that reference this file.

Answer: C

Resume analysis session with fork_session enabled, creating a separate branch for each testing strategy. This preserves the full context of the 23-file

analysis in both branches, allows independent exploration of each testing approach without interference, and avoids the cost of re-reading all source files.

Q196. During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

- A) Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.
- B) Create summaries of all source files before exploration begins, loading only these compressed representations into context.
- C) Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.
- D) Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

Answer: A

Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions. A scratchpad/memory file persists key findings outside the context window, allowing the agent to reference accumulated knowledge without relying on the full conversation history fitting in context. This directly addresses context window limitations in long sessions.

Q197. You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like eval() across a large codebase, which tool should your agent use for content search?

- A) Use Bash to run `ls -R | grep eval` to recursively list files containing eval.
- B) Use Grep to search for the pattern `?eval(?` across all files in the codebase.
- C) Read the project's main entry file and follow import statements to trace where eval might be used.
- D) Use Glob with a pattern like `**/*eval*` to find files, then Read each matching file.

Answer: B

Use Grep to search for the pattern `?eval(?` across all files in the codebase. Grep is the purpose-built tool for content search ? finding specific text patterns across files. It directly searches file contents for the `?eval(?` pattern, which is exactly what's needed for a security scan. Option A uses `ls` which only lists filenames, not file contents; C is impractical for large codebases and would miss dynamically imported files; D searches by filename pattern rather than file content.

Q198. Your agent has spent 25 minutes exploring a game engine's rendering subsystem?reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference ?typical rendering patterns? rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier. What's the most effective approach?

- A) Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- B) Continue in the current context with more targeted prompts referencing the specific classes by name.
- C) Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.
- D) Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.

Answer: D

Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context. The agent is showing signs of context degradation (referencing ?typical patterns? instead of specific classes), so continuing in the same context won't help. Summarizing key findings and passing them to a fresh sub-agent preserves the important knowledge while giving fresh context capacity for the physics exploration.

Q199. An engineer sees an unfamiliar error message ?SYNC_CONFLICT: entity version mismatch detected? in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- A) Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- B) Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- C) Use Grep to search for distinctive text from the error message (like `?SYNC_CONFLICT?` or `?entity version mismatch?`), then Read the matching files to understand context.
- D) Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.

Answer: C

Use Grep to search for distinctive text from the error message (like `?SYNC_CONFLICT?` or `?entity version mismatch?`), then Read the matching files to understand context. Grep is the most efficient tool for locating specific text strings across a large codebase. Searching for the distinctive error message text directly pinpoints the source file(s) without needing to understand the architecture first or guess at directory structures.

Q200. An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module). What exploration strategy will most reliably identify all callers?

- A) Use Grep to search for the function's original name across the codebase.
- B) Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- C) Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.
- D) Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.

Answer: C

Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase. Since the function is re-exported under different names through wrapper modules, you must first discover all aliases by reading the library and wrappers, then search for each name. Searching only the original name (A) would miss renamed usages, and searching imports (B) doesn't account for the renamed exports.

Q201. After adding an MCP server with specialized code refactoring tools (`extract_function` , `rename_variable` , `inline_function`), you notice the agent still uses basic text manipulation via `Write` and `Bash sed` commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like `?extract_function: Extracts a function from code.? What's the most effective way to improve adoption of the MCP refactoring tools?`

- A) Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- B) Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.
- C) Remove the `Write` tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.
- D) Accept this as expected behavior since simpler tools like `sed` are more predictable than specialized refactoring tools.

Answer: A

Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs. The agent selects tools based on their descriptions. Minimal descriptions don't give the agent enough information to understand when the specialized MCP tools are better than generic text manipulation. Enriching descriptions with use cases, expected inputs/outputs, and advantages over alternatives helps the agent make better tool selection decisions.

Q202. Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

- A) Resume the session and immediately have the agent re-read all 12 previously analyzed files
- B) Resume the session without informing the agent about the changed files
- C) Start a fresh session to ensure the agent works with current codebase state without stale assumptions
- D) Resume the session and inform the agent which specific files changed for targeted re-analysis

Answer: D

Resume the session and inform the agent which specific files changed for targeted re-analysis. This balances efficiency (preserving the hour of accumulated context and understanding) with accuracy (ensuring the agent knows which 3 files have stale information and need re-reading). Re-reading all 12 files wastes effort on unchanged files, while not informing about changes risks stale assumptions.

Q203. An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- A) Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.
- B) Read all files containing ?auth?, ?login?, ?permission?, or ?token? in their content or filename.
- C) Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.
- D) Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.

Answer: A

Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally. With 800+ files and context limits, you can't read everything. The most effective strategy is to start with targeted Grep searches to find entry points (login endpoints, auth middleware), read those key files, then incrementally follow the code flow through imports and function calls to build understanding layer by layer.

Q204. An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

- A) Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- B) Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.
- C) Use Grep to search for ?invalidate? and ?expire? patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- D) Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Answer: D

Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations. With 15 files and ~8,000 lines, loading everything would overwhelm context. The most effective approach is to start with the architectural foundation (base cache class/interface), understand the contract, then selectively trace the specific invalidation logic relevant to the task. This builds targeted understanding while managing context.

Q205. Your agent includes an `update_game_score` tool that accepts `game_date` (string), `home_team` (string), and `away_team` (string) parameters. Production logs reveal recurring issues: the agent uses team nicknames instead of official names, applies inconsistent date formats, and selects the wrong game when teams have rematches in the same season. What tool interface change would most effectively prevent these errors?

- A) Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter.
- B) Replace the three parameters with a single `game_id` parameter and a separate `search_games` lookup tool that returns matching game IDs.
- C) Add a season parameter to disambiguate rematches, and add a `confirm_before_update` flag that returns the resolved game details for the agent to verify before the score is committed.
- D) Add detailed examples to the tool description showing the required date format and complete list of official team names.

Answer: B

Replace the three parameters with a single `game_id` parameter and a separate `search_games` lookup tool that returns matching game IDs. This eliminates all three error categories at once: the search tool handles disambiguation (rematches), returns canonical game identifiers (no nickname issues), and removes the need for date formatting. Using a unique `game_id` makes it impossible to select the wrong game, while the search tool provides a structured way to find the correct ID.

Q206. Your agent has a `log_workout` tool that accepts `exercise_type` (string), `value` (number), and `measurement` (string). Production monitoring shows the agent frequently passes mismatched combinations?using `measurement: "reps"` for cardio exercises like running, or `measurement: "miles"` for strength exercises like bench press. Your exercises naturally divide into two categories: cardio (measured in time or distance) and strength (measured in reps and sets). 23% of tool calls have invalid combinations. What approach would most effectively reduce these errors?

- A) Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections.
- B) Split into `log_cardio_workout` (with `duration_minutes` or `distance_miles` parameters) and `log_strength_workout` (with `reps` and `sets` parameters).
- C) Add enum constraints on `measurement` limiting values to `?minutes?`, `?miles?`, `?reps?`, or `?sets?` to prevent arbitrary measurement strings.
- D) Add explicit examples to the tool description showing valid combinations (e.g., `?For running: use minutes or miles. For push-ups: use reps?`) with constraints for each exercise category.

Answer: B

Split into `log_cardio_workout` (with `duration_minutes` or `distance_miles` parameters) and `log_strength_workout` (with `reps` and `sets` parameters). Splitting into two purpose-specific tools makes invalid combinations structurally impossible. The tool schema itself enforces correct parameter combinations ? cardio tools only accept time/distance measurements, strength tools only accept reps/sets. This eliminates the mismatch problem at the design level rather than relying on the agent to remember valid c

Q207. Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a `search_connectors(description)` tool that finds matching connectors, but in testing, agents frequently skip searching and call connectors directly (often incorrectly), or search correctly but still select wrong connectors from the filtered results. How should you design the tool composition pattern to address both issues?

- A) Design connectors with built-in compatibility validation that returns descriptive errors for mismatched requests.
- B) Design a `find_and_execute(description, params)` composite tool that searches and immediately executes the best-matching connector.
- C) Design `search_connectors` to dynamically add matched connectors to the agent?s available tools. Connectors start unavailable and persist once discovered.
- D) Enhance all connector descriptions with detailed usage examples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.

Answer: C

Design `search_connectors` to dynamically add matched connectors to the agent?s available tools. Connectors start unavailable and persist once discovered. This addresses both issues: agents can?t skip searching because connectors aren?t available until discovered, and once found through search, the correct connector is added to the tool set for direct use. This enforces the search-first pattern while still allowing efficient subsequent calls.

Q208. Your `update_user_profile` tool accepts a `user_id` (required) and an optional `fields_to_update` object. In testing, Claude frequently omits `user_id` or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

- A) Verbose parameter names encoding format hints, such as `user_id_string_uuid_format`
- B) Strict JSON Schema type constraints marking `user_id` as required and defining `fields_to_update` as an object type
- C) Clear parameter descriptions explaining expected format, such as `?user_id: UUID of the user to update (required)?`
- D) Detailed error responses explaining why invalid parameter values were rejected

Answer: B

Strict JSON Schema type constraints marking `user_id` as required and defining `fields_to_update` as an object type. JSON Schema type constraints are the most critical mechanism for helping Claude understand parameter requirements. Marking fields as required ensures Claude knows it must provide them, and defining proper types (object, string, etc.) gives Claude structural guidance on what values to pass. This is enforced at the schema level before the tool is even called.

Q209. You're deciding whether to expose your weather API integration as an MCP server or implement it as a custom tool directly in your agent application. Your team plans to build several different AI applications that will all need weather data. What is the primary advantage of the MCP approach?

- A) MCP provides built-in retry logic that automatically handles failed API calls with exponential backoff.
- B) Any MCP-compatible client can connect to your weather server without writing custom integration code.
- C) MCP servers process tool calls faster than custom implementations because they use an optimized binary protocol.
- D) MCP automatically handles authentication and rate limiting with the weather API, reducing backend implementation work.

Answer: B

Any MCP-compatible client can connect to your weather server without writing custom integration code. The primary advantage of MCP is interoperability and reusability. By exposing the weather API as an MCP server, any MCP-compatible client (all the different AI applications your team is building) can connect to it using the standard protocol without needing custom integration code for each application. This is the core value proposition of MCP ? write the integration once, use it everywhere.

Q210. Your send_notification tool calls third-party messaging APIs. When these services time out during delivery, you cannot determine whether the message was actually sent. Currently, the tool returns is_error: true with a generic ?Notification failed? message for all timeouts. Production monitoring reveals agents automatically retry these failures, frequently causing users to receive duplicate notifications. How should you modify the error response?

- A) Return is_error: true with a message communicating uncertainty: ?Timeout?status unknown. Message may have been sent. Avoid retry.?
- B) Return is_error: false with the original message content echoed back.
- C) Return is_error: true with a message encouraging retry: ?Delivery service temporarily unavailable. Please retry the notification.?
- D) Return is_error: true with a structured field retry_safe: truefor timeouts, distinguishing them from permanent failures that should not be retried.

Answer: A

Return is_error: true with a message communicating uncertainty: ?Timeout?status unknown. Message may have been sent. Avoid retry.? The key issue is that timeouts are ambiguous ? the message may have been delivered. The error response must communicate this uncertainty to prevent the agent from retrying and causing duplicates. Explicitly stating ?Avoid retry? in the message guides the agent?s behavior correctly.

Q211. Your shipment tracking tool queries multiple carriers (FedEx, UPS, DHL) that each return status information in different formats?FedEx uses numeric codes, UPS uses descriptive phrases, and DHL returns timestamped event arrays. The agent uses these results to determine delivery status and escalate delayed shipments. How should you structure the tool?s return value to best enable the agent?s reasoning?

- A) Design separate tool endpoints for each carrier (track_fedex, track_ups, track_dhl) with carrier-specific response schemas.
- B) Return a normalized schema with status, estimated_delivery, delay_reason, and requires_action fields, converting carrier formats internally.
- C) Return both a normalized summary and the complete raw carrier response in each tool call.
- D) Return raw carrier responses with source metadata, encoding carrier-specific interpretation logic in the system prompt.

Answer: B

Return a normalized schema with status, estimated_delivery, delay_reason, and requires_action fields, converting carrier formats internally. Normalizing the response server-side gives the agent a consistent, predictable schema to reason about regardless of carrier. The agent doesn?t need to understand carrier-specific formats ? it just needs actionable fields like status, estimated delivery, and whether action is required. This simplifies the agent?s reasoning and reduces errors from misinterpre

Q212. Your CRM agent?s delete_contact tool handles requests like ?delete the duplicate entry for Acme Corp.? The database contains similarly named records (e.g., ?Acme Corp.,? ?Acme Corporation,? ?ACME Corp Inc.?), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

- A) Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.
- B) Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names.
- C) Present matched records with differentiating fields and require single-click confirmation of the intended target before executing

deletion.

D) Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.

Answer: C

Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion. This balances both requirements: it reduces errors by showing the user disambiguating information (so they can verify the correct record), while maintaining efficiency with a single-click confirmation rather than a multi-step flow. It addresses the root cause (misidentification among similar names) without adding excessive friction.

Q213. Your `search_products` tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the current design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?

- A) Add a `max_pages` parameter (default: 2) that controls how many pages are fetched internally.
- B) Implement server-side relevance ranking and return only the top 50 most relevant items.
- C) Create separate `search_products` and `fetch_more_results` tools for pagination.
- D) Return the first page with total match count and cursor for additional pages.

Answer: D

Return the first page with total match count and cursor for additional pages. This gives the agent immediate results (no delay), awareness of how many total results exist, and the ability to fetch more if needed via the cursor. It puts the agent in control of whether to paginate further based on whether the first page satisfies the user's needs, avoiding unnecessary fetches while maintaining access to all results.

Q214. Your MCP server implements a `check_availability` tool that queries an external calendar API. During testing, you encounter three error conditions: (1) the tool is called with a malformed request missing the required `user_email` parameter, (2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system, and (3) the calendar API returns a 503 because the service is temporarily unavailable. How should each error be reported according to MCP's error handling design?

- A) Report errors 1 and 2 as JSON-RPC protocol errors; report error 3 as a tool result with `isError: true`.
- B) Report error 1 as a JSON-RPC protocol error; report errors 2 and 3 as tool results with `isError: true`.
- C) Report all three as JSON-RPC protocol errors.
- D) Report all three as tool results with `isError: true`.

Answer: B

Report error 1 as a JSON-RPC protocol error; report errors 2 and 3 as tool results with `isError: true`. In MCP's error handling design, JSON-RPC protocol errors are for issues with the request itself (malformed requests, missing required parameters) ? these are infrastructure-level problems. Tool results with `isError: true` are for errors that occur during tool execution (user not found, service unavailable) ? these are business-logic or runtime errors the model should see and reason about. Error

Q215. Your `track_shipment(tracking_id)` tool queries an external logistics API that sometimes fails?the API may be temporarily unavailable, the tracking ID may be malformed, or the shipment may not exist. Currently, your tool raises a Python exception when errors occur. Users report the agent gives unhelpful responses like `?I'm having trouble with that request?` instead of suggesting alternatives such as verifying the tracking number format or checking by order number. How should you handle errors in tool results?

- A) Return a generic error response (e.g., `{"success": false, "error": "lookup_failed"}`) for all failure cases to maintain a consistent schema and avoid exposing internal error details.
- B) Implement retry logic with exponential backoff inside the tool implementation so transient errors are automatically handled and only return a result after all retry attempts are exhausted.
- C) Return structured error information as normal tool output including error type, recoverability status, and actionable context for the user.
- D) Create dedicated error-recovery tools (`retry_tracking_lookup`, `search_by_order_number`) that the model can invoke after the primary tracking tool returns a failure indicator.

Answer: C

Return structured error information as normal tool output including error type, recoverability status, and actionable context for the user. By returning errors as structured data (not exceptions), the agent receives rich context about what went wrong, whether it's recoverable, and what alternatives exist. This enables the agent to provide helpful responses like `?The tracking ID format appears invalid ? did you mean??` or `?The API is temporarily down, try again in a few minutes?` rather than generi

Q216. Your content curation agent discovers articles, analyzes each for relevance, then adds selected articles to themed collections. With separate `discover_articles(topic)`, `analyze_article(id)`, and `add_to_collection(article_id, collection_id)` tools, you observe 18+ sequential tool calls per request, causing latency issues. The agent must make editorial judgments about which articles fit a collection's theme; this requires seeing all candidates with their analysis scores simultaneously to select a cohesive set. What tool composition best addresses efficiency while preserving editorial judgment?

- A) Keep all tools separate but implement response caching for `analyze_article` calls.
- B) Add a `preview_curation(topic, collection_id)` tool that shows what would be added based on predefined rules, with an `approve_curation()` tool to confirm.
- C) Create a `discover_and_analyze(topic)` composite tool that returns all candidates with their analysis scores, keeping `add_to_collection` separate for selective calls.
- D) Create a `curate_collection(topic, collection_id)` tool that handles discovery, analysis, and selection internally using configurable quality thresholds.

Answer: C

Create a `discover_and_analyze(topic)` composite tool that returns all candidates with their analysis scores, keeping `add_to_collection` separate for selective calls. This combines the discovery and analysis steps (reducing many sequential calls to one) while preserving the agent's editorial judgment by presenting all candidates with scores simultaneously. The agent can then make informed selection decisions and call `add_to_collection` only for chosen articles. Option D removes editorial judgment en

Q217. Production monitoring shows your `search_catalog` tool fails 12% of the time: 8% are network timeouts that succeed when immediately retried, while 4% are query syntax errors from malformed user-provided filters that never succeed regardless of retry attempts. Currently, both error types are returned to the agent identically, causing it to waste turns retrying syntax errors and telling users to "try again later" for timeouts. How should you modify the tool's error handling?

- A) Apply exponential backoff retry logic to all errors uniformly, returning a generic "service temporarily unavailable" message after max retries are exhausted.
- B) Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details.
- C) Add few-shot examples to your system prompt demonstrating how to distinguish network errors from syntax errors and handle each case appropriately.
- D) Return all errors with a retryable boolean flag and error type details.

Answer: B

Implement automatic retry with backoff for network timeouts inside the tool; return syntax errors immediately with parameter validation details. This handles each error type optimally at the right layer: transient network timeouts are retried automatically within the tool (the agent never sees them if they succeed), while syntax errors are returned immediately with validation details so the agent can fix the query. Option A wastes time retrying unfixable errors; C relies on fragile prompt-based

Q218. Your agent uses three tools: `get_property_details(property_id)` returns data including street address, `get_price_history(property_id)` returns historical pricing, and `get_neighborhood_info(address)` returns area statistics. You observe that `get_neighborhood_info` always requires calling `get_property_details` first just to extract the address, even when users specify the property by ID. This creates unnecessary latency and failure coupling; if the first call fails, the neighborhood request also fails. What tool design change best addresses this?

- A) Consolidate into a single `get_property_with_neighborhood(property_id)` tool returning both datasets.
- B) Change `get_neighborhood_info` to accept `property_id`, resolving the address internally.
- C) Create a `lookup_address(property_id)` helper tool for retrieving addresses.
- D) Add retry logic and timeout handling to `get_property_details`.

Answer: B

Change `get_neighborhood_info` to accept `property_id`, resolving the address internally. This eliminates the coupling by allowing the agent to call `get_neighborhood_info` directly with the property ID, removing the need for a separate `get_property_details` call just to get the address. The address resolution happens server-side internally. Option A over-consolidates tools that serve different purposes; C adds another tool call rather than reducing them; D doesn't address the fundamental coupling issue

Q219. Your resource allocation tool returns a simple acknowledgment message after provisioning is requested. Users frequently approve allocations and immediately ask "how much did that cost?" or "which project was that?" - indicating they confirmed without understanding the request. What tool design change would most effectively address this?

- A) Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response.
- B) Add a user_acknowledged boolean parameter that must be set true, with instructions for the agent to only set it after the user explicitly confirms they reviewed the details.
- C) Add a detail_level parameter with options "minimal" or "comprehensive" that controls how much context the agent presents in confirmations.
- D) Implement a 60-second hold before execution completes, allowing users time to review pending allocations and cancel if needed.

Answer: A

Return structured data including cost estimate, target project, resource specifications, and impact summary in the tool response. The problem is that users confirm without understanding what they're approving. By returning structured data (cost, project, resources, impact) in the tool response, the agent can present this information to the user before or during confirmation, ensuring informed consent. Option B adds friction but doesn't guarantee the user actually understands; C controls presenta

Q220. Your process_refund tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 30 day return window", "Item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude. What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- A) Add a check_refund_eligibility tool that must be called before process_refund to prevent business rule violations.
- B) Return structured error responses with retrieable: false for business errors and a customer-friendly explanation for Claude to use.
- C) Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.
- D) Add few-shot examples showing how to distinguish retrieable from non-retrieable errors by parsing error message text.

Answer: B

Return structured error responses with retrieable: false for business errors and a customer-friendly explanation for Claude to use. This directly addresses both problems: the retrieable: false flag tells Claude not to retry business errors (eliminating wasted turns), and the customer-friendly explanation improves response quality. Option C handles retries at the tool level but doesn't help Claude communicate the reason to customers. Option A adds unnecessary complexity with a pre-check tool. Optio

Q221. After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call escalate_to_human, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- A) The complete conversation transcript with all tool results.
- B) Your diagnosis and the refund amount only.
- C) A structured summary: customer ID, root cause, refund amount, and recommended action.
- D) The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

Answer: C

A structured summary: customer ID, root cause, refund amount, and recommended action. A structured summary provides the human agent with everything they need to act quickly without wading through 25+ turns of conversation. It includes the key identifiers (customer ID), the diagnosis (root cause), the specific action needed (refund amount), and a recommendation. The full transcript (A) is too verbose; diagnosis and amount only (B) lacks customer identification; verbatim complaint plus excerpts (D)

Q222. The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- A) The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- B) Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
- C) The verification tool is clearing the agent's internal state after each successful validation step.
- D) The conversation history isn't being passed in subsequent API requests.

Answer: D

The conversation history isn't being passed in subsequent API requests. Claude doesn't have persistent memory between API calls ? it only knows what's included in the messages array. If the application isn't appending previous turns to subsequent requests, Claude will have no knowledge of earlier exchanges. This is the most common cause of ?amnesia? in multi-turn conversations. Option A is wrong because Claude naturally uses context provided; B is wrong because there's no such two-turn limit; C

Q223. Production logs reveal inconsistent error handling: when lookup_order fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: {"isError": true, "content": [{"type": "text", "text": "Operation failed"}]} . The agent cannot distinguish between error types. What's the most effective improvement ?

- A) Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.
- B) Enhance error responses with structured metadata: include errorCategory (transient/validation/permission), isRetryable boolean, and a description of what caused the failure.
- C) Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.
- D) Create an analyze_error MCP tool the agent calls after any failure to determine the error category and recommended action.

Answer: B

Enhance error responses with structured metadata: include errorCategory (transient/validation/permission), isRetryableboolean, and a description of what caused the failure. The root cause is that the agent receives uniform, undifferentiated error responses. By adding structured metadata like error category and retryability, the agent can make informed decisions about whether to retry, escalate, or ask for clarification. This is more efficient than a separate analysis tool (D), more reliable than

Q224. A customer sends: ?This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW.? The agent has not yet called any tools to investigate their account. What should the agent do?

- A) First call get_customer and lookup_order to gather account context, then escalate to a human agent.
- B) Immediately call escalate_to_human with the conversation history.
- C) Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.
- D) Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.

Answer: B

Immediately call escalate_to_human with the conversation history. When a customer explicitly and firmly requests to speak to a human, the agent should honor that request immediately. The customer has already expressed frustration about repeating themselves, so asking more questions (C) or trying to retain them (D) would increase frustration. Gathering account context first (A) adds unnecessary delay when the customer has made a clear, direct request for escalation.

Q225. A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks ?What happened with my refund?? The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- A) Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.
- B) Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- C) Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.
- D) Implement sliding window context that retains the most recent 30 turns.

Answer: C

Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer. Structured issue data in a separate context layer ensures all three issues remain accessible regardless of context window limits. Unlike a sliding window (D) which would lose early turns, or summaries (B) which lose precision, structured data preserves the exact details needed to address any issue when the customer references it again. MCP re-fetching (A) might work but doesn't preserve conver

Q226. Your agent has called lookup_order multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

- A) Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions
- B) Proceed with additional lookups without modifying the existing tool output context
- C) Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details
- D) Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

Answer: C

Extract only return-relevant fields (items, purchase date, return window, status) from each existing order response, removing verbose details. Since the conversation is about returns, only return-relevant fields matter. Extracting just those fields from existing responses frees up context space for the two additional order lookups while preserving the information needed for the task. This is more precise than prose summaries (A) and more practical than a vector database (D).

Q227. During a billing dispute resolution, your agent successfully retrieves customer info via get_customer and order details via lookup_order, but when attempting to call process_refund, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

- A) Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically
- B) Implement automatic retries with exponential backoff for process_refund, keeping the conversation open until the refund is successfully processed
- C) Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later
- D) Escalate immediately to a human agent since the refund action cannot be completed

Answer: C

Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later. This maximizes first-contact value by providing all the information the agent can (billing explanation, eligibility confirmation), is transparent about the system limitation (timeout), and gives the customer options (escalation or retry). It doesn't make false promises (A), doesn't keep the customer waiting indefinitely (B), and doesn't escalate prematurely.

Q228. A customer writes: 'I've been going back and forth on this return for days. I just want to speak to someone who can actually help me.' The agent has confirmed via lookup_order that the return is straightforward, within policy and eligible for immediate processing. What should the agent do?

- A) Process the refund via process_refund to resolve the underlying issue, then inform them it's complete
- B) Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically
- C) Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate
- D) Call escalate_to_human immediately to honor the customer's request

Answer: C

Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate. The customer is frustrated and explicitly asked for a human. The best approach acknowledges their frustration (showing empathy), informs them the issue is actually simple and resolvable right now (providing useful context), and gives them the choice to complete it immediately or escalate as requested. This respects the customer's autonomy while offering a faster resolution path.

Q229. Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent; this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- A) Modify the refund tool to return an error with message 'Amount exceeds policy limit; please escalate' when threshold is exceeded.
- B) Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.

C) Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).

D) Strengthen the system prompt with emphatic language: ?CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly.?

Answer: B

Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation. For guaranteed compliance, you cannot rely on model behavior (which already has a 3% failure rate). A programmatic hook that intercepts tool calls provides a deterministic enforcement layer ? it physically prevents the refund from being processed regardless of what the model decides. Option A still allows the model to attempt the call; C and D are prompt-based approaches t

Q230. Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order` , it identifies that the dispute involves a promotional pricing error requiring manager approval?beyond the agent?s authorization level. How should the workflow handle this mid-process escalation?

A) Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.

B) Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.

C) Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.

D) Call `escalate_to_human` passing only the customer?s original message.

Answer: A

Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`. A structured handoff gives the human agent everything they need to act immediately: who the customer is, what the order details are, and what the specific issue is (promotional pricing error needing manager approval). Option B adds unnecessary infrastructure complexity; C attempts an unauthorized action which could cause issues; D loses all the diagnostic work already done,

Q231. A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing ?Status: PENDING. Expected resolution: 24-48 hours.? In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., ?I see your refund is still being processed?) even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

A) Resume with full history but filter out previous `tool_result` messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.

B) Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.

C) Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

D) Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.

Answer: C

Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging. Starting fresh with a structured summary eliminates the stale data problem entirely ? there are no outdated tool results in context to confuse the agent. The summary preserves important context (what happened, what was decided) without carrying stale state. Option A risks losing conversational context; B wastes resources re-calli

Q232. You?re implementing the escalation logic for when the agent should call `escalate_to_human` . Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

A) Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer?s stated issue, ensuring a reasonable attempt before involving a human.

B) Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.

C) Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

D) Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

Answer: C

Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make

meaningful progress. This approach covers the three genuine cases requiring human intervention: explicit customer request (respecting autonomy), policy exceptions (beyond agent authority), and inability to progress (avoiding loops). Option A uses an arbitrary threshold that may escalate too late or too early; B is rigid and can't handle novel situations; D r

Q233. When implementing your lookup_order MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- A) Return the error message in the tool result content with the isError flag set to true.
- B) Return a success response with a "status" field indicating the error type.
- C) Throw an exception from the tool handler so the agent framework can catch and log it.
- D) Log the error server-side and return an empty result to avoid confusing the model.

Answer: A

Return the error message in the tool result content with the isError flag set to true. The MCP protocol specifies that tool execution errors should be communicated by setting isError: true in the response and including the error description in the content. This allows the model to understand what went wrong and respond appropriately (e.g., inform the user, try a different approach). Option B conflates success and error states; C breaks the MCP protocol contract; D hides information the model nee

Q234. When the agent calls lookup_order and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call process_refund or escalate_to_human next?

- A) The orchestration layer automatically routes to the next tool based on the order's status field.
- B) The agent executes the remaining steps in a tool sequence planned at the start of the request.
- C) The order details are added to the conversation and the model reasons about which action to take.
- D) The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

Answer: C

The order details are added to the conversation and the model reasons about which action to take. In an agentic loop, tool results are appended to the conversation context, and the model uses reasoning to decide the next action based on all available information. The model considers the 45-day purchase age against return policies, customer context, and system prompt instructions to determine whether to process the refund or escalate. This is the fundamental nature of agentic systems ? the model